

Signals and Systems

#Continuous-Time and Discrete-Time Sinusoidal Signals

This notebook demonstrates the graphical representation of continuous-time (CT) and discrete-time (DT) sinusoidal signals using Python.

The objective is to understand:

- CT signal representation
- DT signal representation
- Difference between continuous and discrete signals
- Effect of sampling

Aim- 1

1. Generate a continuous-time sine wave.
2. Generate a discrete-time sine wave.
3. Compare CT and DT representations graphically.

```
import numpy as np
import matplotlib.pyplot as plt
```

Continuous-Time Sinusoidal Signal

```
# Parameters

A = 1          # Amplitude
f = 5          # Frequency in Hz

# Continuous time axis
t = np.arange(0, 1, 0.001)

# Continuous-time signal
x_ct = A * np.sin(2 * np.pi * f * t)

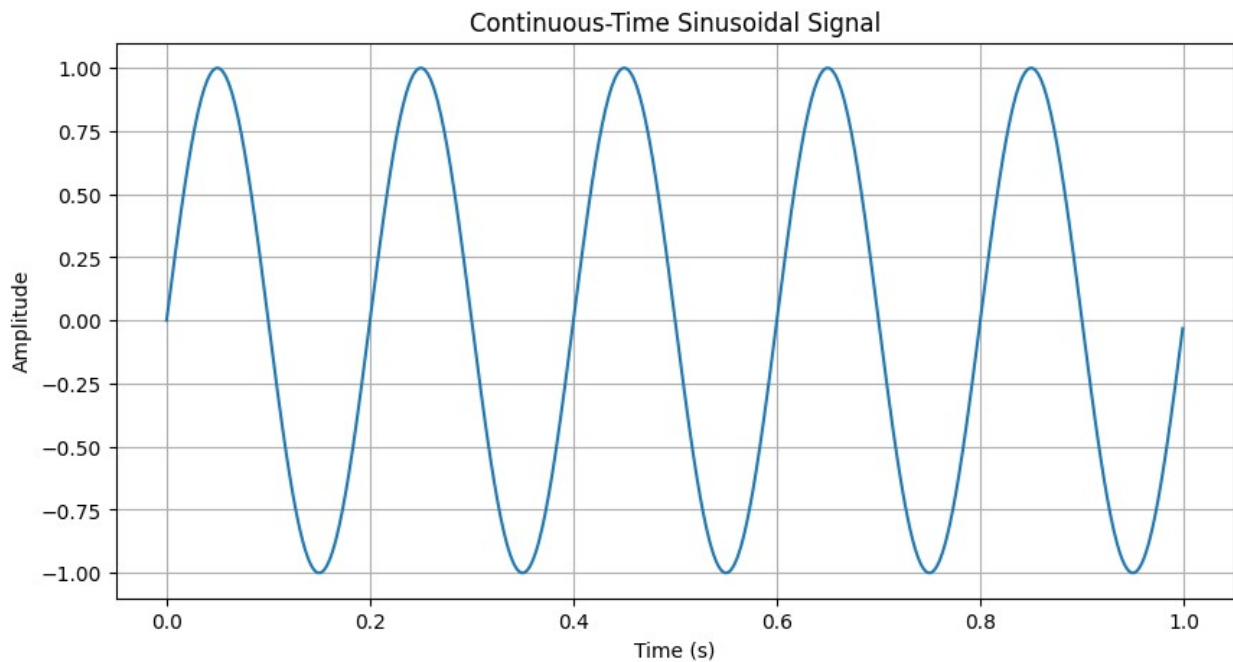
# Plotting
plt.figure(figsize=(10,5))
```

```
plt.plot(t, x_ct)

plt.title("Continuous-Time Sinusoidal Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")

plt.grid()

plt.show()
```



Observation

1. The signal varies continuously with time.
2. The waveform completes 5 cycles within 1 second.
3. The amplitude varies smoothly between +1 and -1.

Discrete-Time Sinusoidal Signal

```
# Discrete-time index
n = np.arange(0, 20)

# Discrete angular frequency
omega = np.pi / 4
```

```
# Discrete-time signal
x_dt = A * np.sin(omega * n)

# Plotting

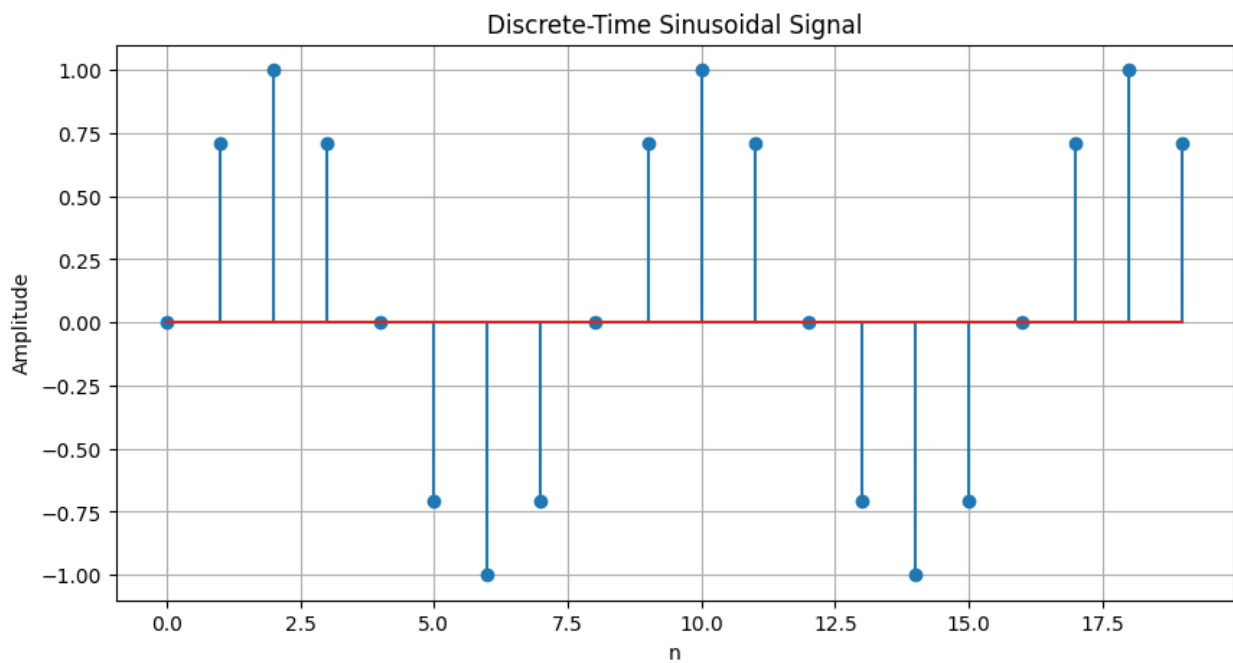
plt.figure(figsize=(10,5))

plt.stem(n, x_dt)

plt.title("Discrete-Time Sinusoidal Signal")
plt.xlabel("n")
plt.ylabel("Amplitude")

plt.grid()

plt.show()
```



Observation

1. The signal exists only at discrete integer values of n .
2. Stem plots are used for discrete-time signals.
3. Unlike CT signals, DT signals are not defined between sample points.

Comparison Between CT and DT Signals

Continuous-Time Signal	Discrete-Time Signal
Defined for all time instants	Defined only at integer values
Uses continuous variable t	Uses discrete index n
Plotted using line graphs	Plotted using stem plots
Smooth waveform	Sampled waveform

Comparison between CT and DT signals

```
# Comparison of CT and DT signals
```

```
plt.figure(figsize=(12,5))
```

```
# Continuous-time plot
```

```
plt.subplot(1,2,1)
```

```
plt.plot(t, x_ct)
```

```
plt.title("CT Signal")
```

```
plt.xlabel("Time")
```

```
plt.ylabel("Amplitude")
```

```
plt.grid()
```

```
# Discrete-time plot
```

```
plt.subplot(1,2,2)
```

```
plt.stem(n, x_dt)
```

```
plt.title("DT Signal")
```

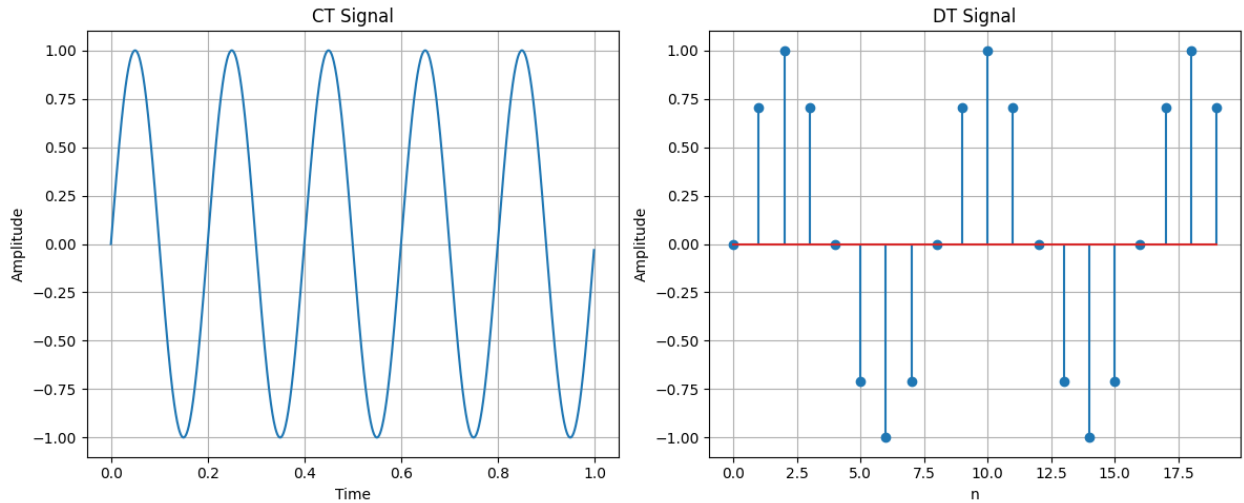
```
plt.xlabel("n")
```

```
plt.ylabel("Amplitude")
```

```
plt.grid()
```

```
plt.tight_layout()
```

```
plt.show()
```



#Energy and Power Signals

Energy of a Continuous-Time Signal

$$E = \int_{-\infty}^{\infty} |x(t)|^2 dt$$

A signal is called an energy signal if:

- Energy is finite
- Average power is zero

Power of a Continuous-Time Signal

$$P = \lim_{T \rightarrow \infty} \frac{1}{2T} \int_{-T}^T |x(t)|^2 dt$$

A signal is called a power signal if:

- Average power is finite
- Energy is infinite

Aim- 2

1. Generate an energy signal.
2. Generate a power signal.
3. Compute signal energy numerically.
4. Compute signal power numerically.
5. Compare energy and power signals graphically.

```
import numpy as np
import matplotlib.pyplot as plt
```

Example 1 — Energy Signal

An exponentially decaying signal is an example of an energy signal.

$$x(t) = e^{-t} u(t)$$

where:

- $u(t)$ is the unit step function.

```
# Time axis
t = np.arange(0,10,0.001)

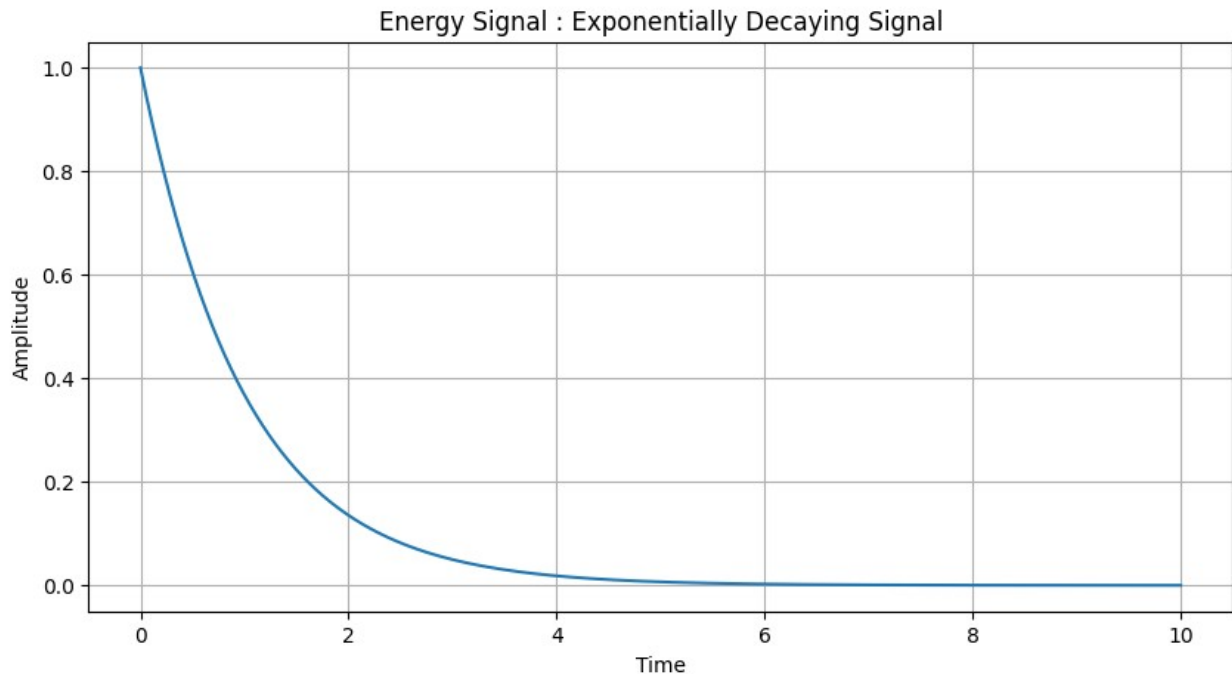
# Energy signal
x1 = np.exp(-t)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,x1)

plt.title("Energy Signal : Exponentially Decaying Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")

plt.grid()

plt.show()
```



Observation

1. The signal amplitude decreases with time.
2. The signal approaches zero as time increases.
3. Such signals possess finite energy.

Numerical Computation of Energy

```
# Energy computation using numerical integration
```

```
E = np.trapezoid(np.abs(x1)**2,t)
```

```
print("Energy of the signal =",E)
```

```
Energy of the signal = 0.5000001656340152
```

Observation

The computed energy is finite.

Hence, the signal is classified as an energy signal.

Example 2 — Power Signal

A sinusoidal signal is an example of a power signal.

$$x(t) = \sin(2\pi f t)$$

```
# Parameters
f = 5

# Time axis
t2 = np.arange(0,10,0.001)

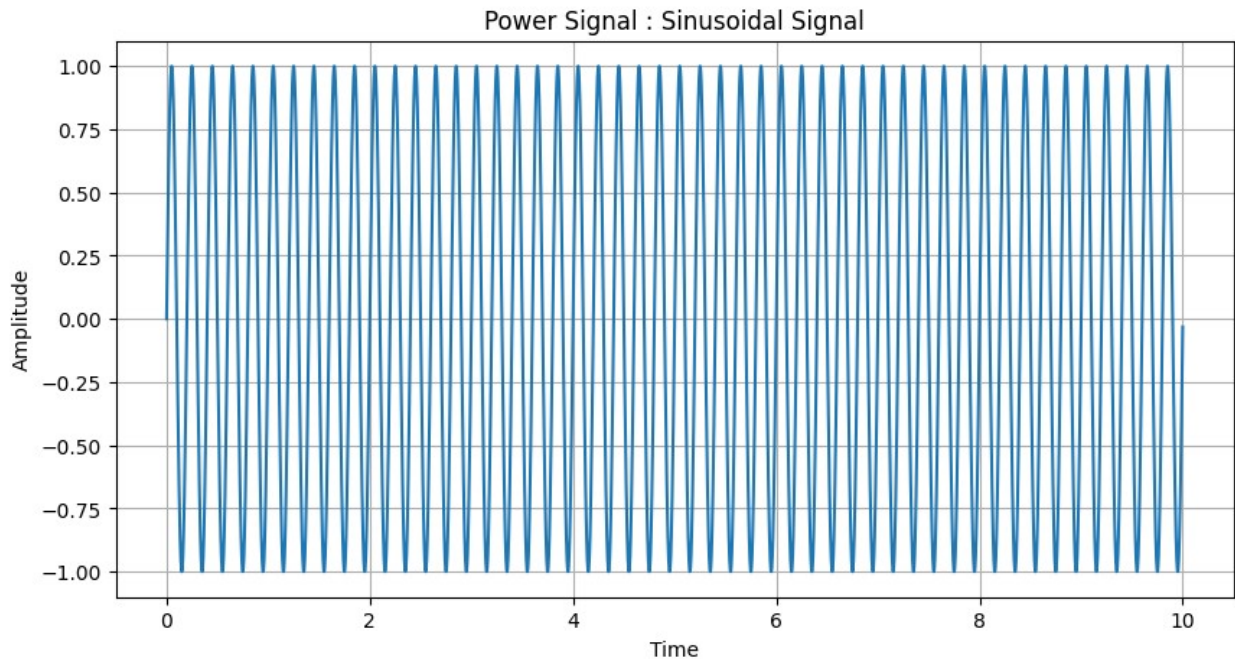
# Power signal
x2 = np.sin(2*np.pi*f*t2)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t2,x2)

plt.title("Power Signal : Sinusoidal Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")

plt.grid()

plt.show()
```



Observation

1. The signal continues indefinitely.
2. The amplitude remains bounded.
3. Such signals possess finite average power.

Numerical Computation of Power

```
# Approximate power computation  
T = 10  
P = (1/(2*T)) * np.trapezoid(np.abs(x2)**2,t2)  
print("Average Power of the signal =",P)  
Average Power of the signal = 0.24999997533410534
```

Observation

The computed average power is finite.

Hence, the sinusoidal signal is classified as a power signal.

Automatic Signal Classification

```
# Automatic classification

if E < 1000 and P < 0.001:
    print("The signal is an Energy Signal")

elif P > 0 and P < 100:
    print("The signal is a Power Signal")

else:
    print("The signal is neither energy nor power signal")

The signal is a Power Signal
```

#Transformation of Independent Variables

Common transformations include:

1. Time Shifting
 2. Time Reversal
 3. Time Scaling
-

Time Shifting

$$x[n-k]$$

- Delay by k units
-

Time Reversal

$$x[-n]$$

- Flips the signal about $n = 0$
-

Time Scaling

$$x[an]$$

- Compresses or expands the signal

Aim- 3

1. Generate an original discrete-time signal.
2. Perform time shifting.
3. Perform time reversal.
4. Perform time scaling.
5. Visualize transformed signals.

```
import numpy as np
import matplotlib.pyplot as plt
```

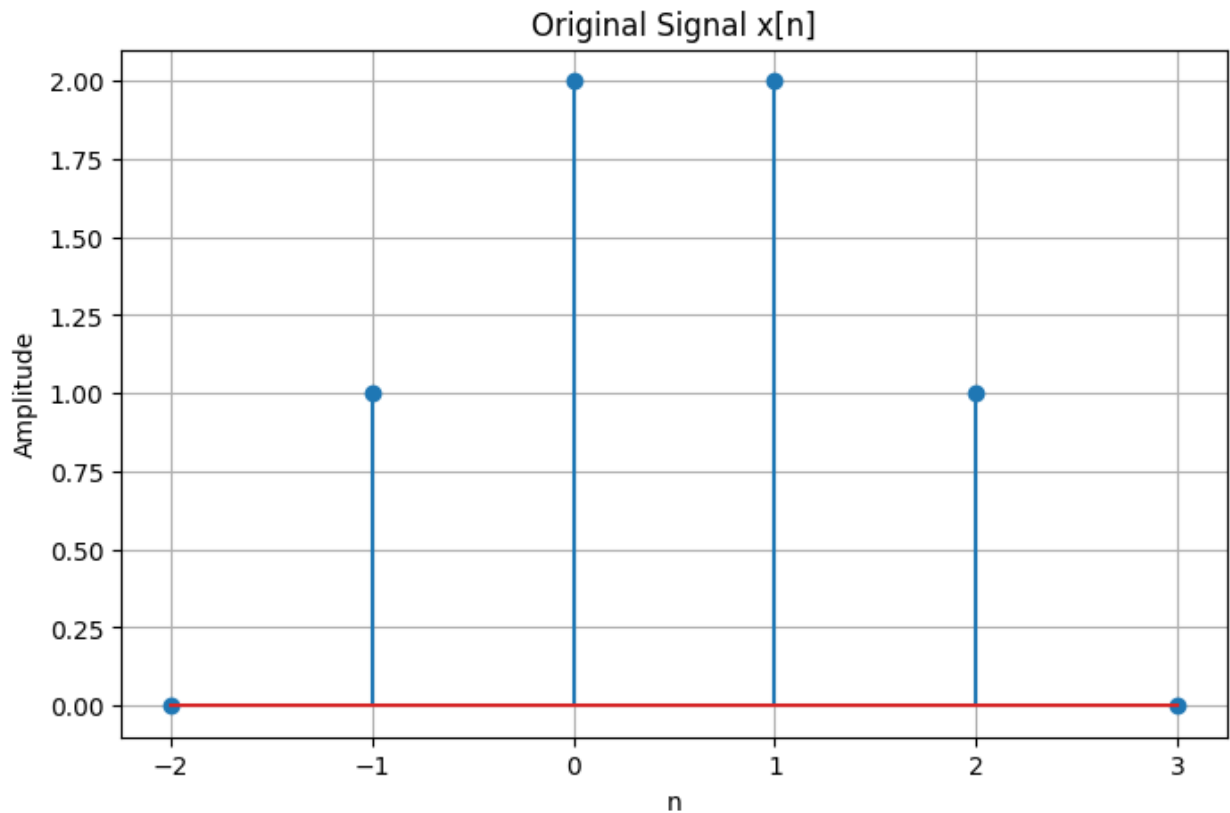
Original Discrete-Time Signal

```
# Original signal
n = np.array([-2,-1,0,1,2,3])
x = np.array([0,1,2,2,1,0])

# Plotting
plt.figure(figsize=(8,5))
plt.stem(n,x)

plt.title("Original Signal x[n]")
plt.xlabel("n")
plt.ylabel("Amplitude")

plt.grid()
plt.show()
```



Time Shifting

A delayed signal is represented as:

$$x[n-2]$$

This shifts the signal to the right by 2 units.

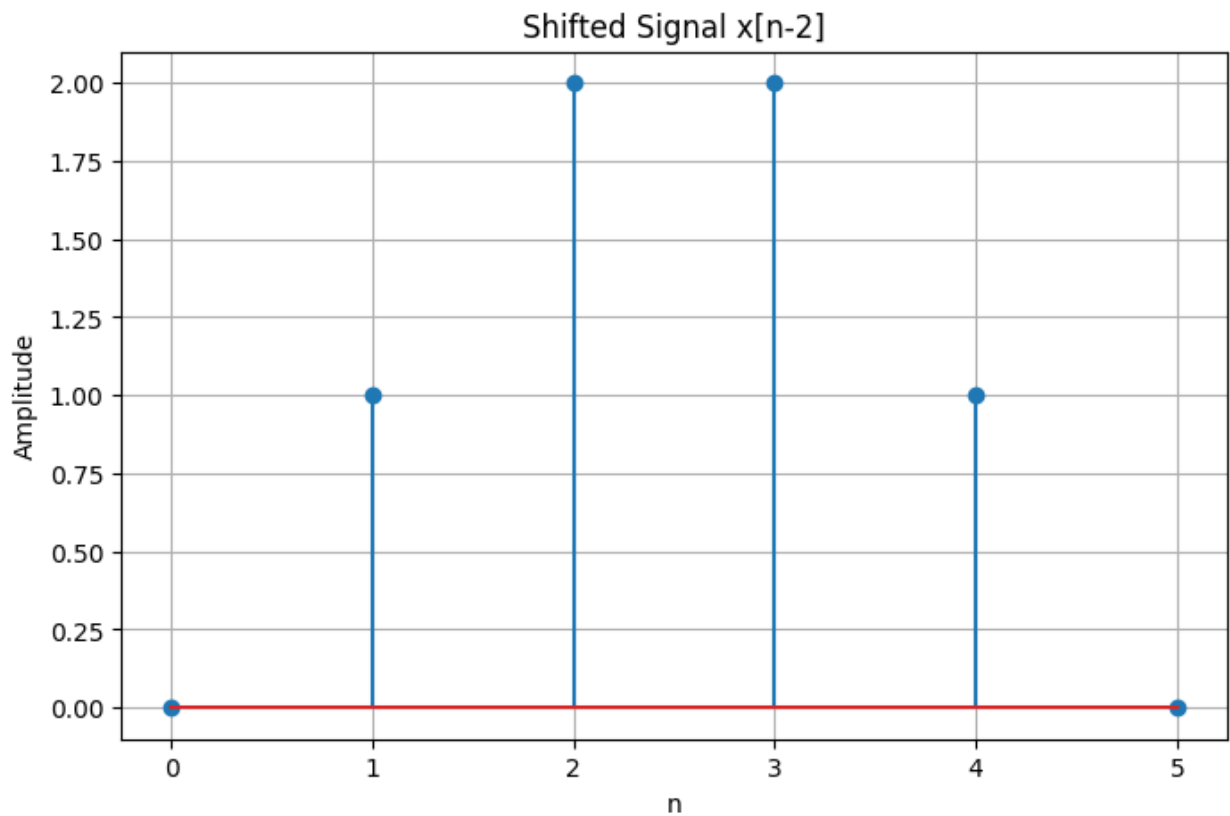
```
# Time shifting
n_shift = n + 2

# Plotting
plt.figure(figsize=(8,5))
plt.stem(n_shift,x)

plt.title("Shifted Signal  $x[n-2]$ ")
plt.xlabel("n")
plt.ylabel("Amplitude")

plt.grid()
```

```
plt.show()
```



Observation

1. The signal shifts to the right.
2. Signal shape remains unchanged.
3. Only the position of the signal changes.

Time Reversal

Time reversal is represented as:

$$x[-n]$$

This flips the signal about $n = 0$.

```
# Time reversal
```

```
n_reverse = -n
```

```
# Plotting

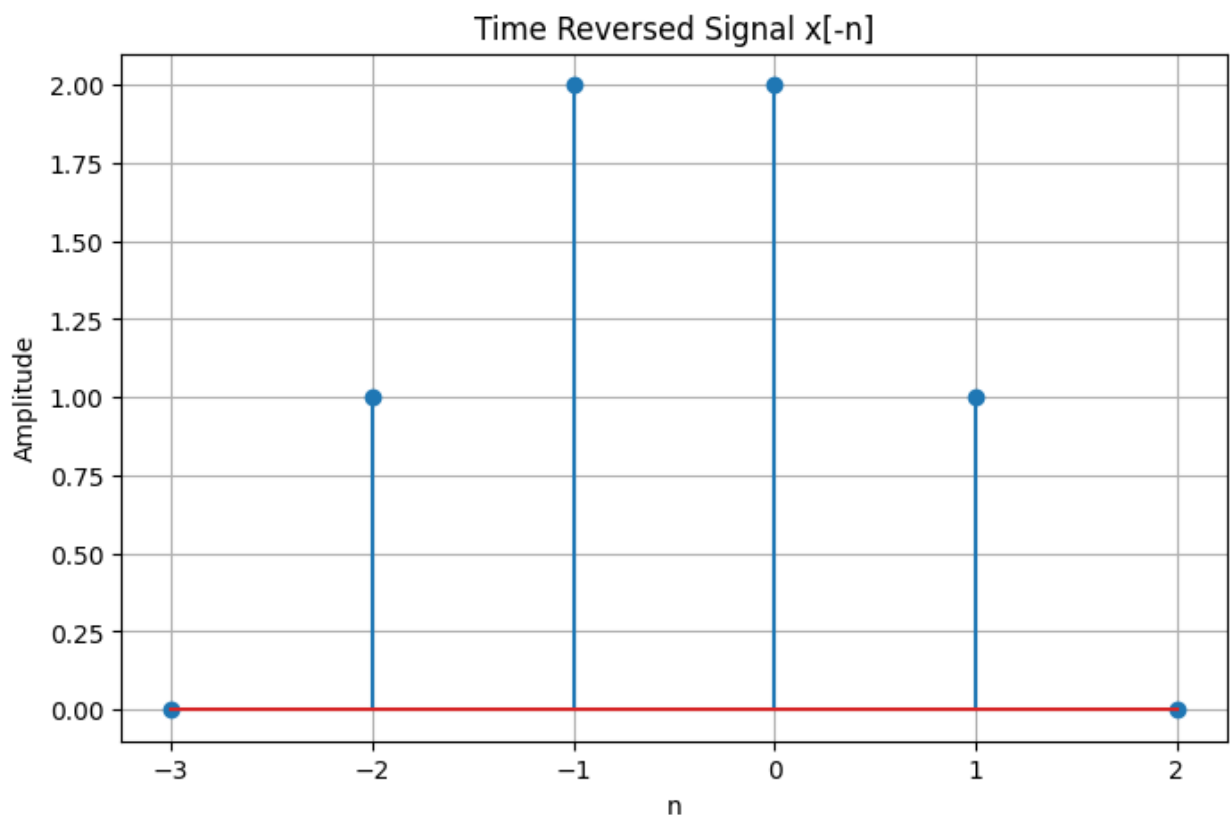
plt.figure(figsize=(8,5))

plt.stem(n_reverse,x)

plt.title("Time Reversed Signal x[-n]")
plt.xlabel("n")
plt.ylabel("Amplitude")

plt.grid()

plt.show()
```



Observation

1. The signal flips about $n = 0$.
2. Left-side samples move to the right.
3. Right-side samples move to the left.

Time Scaling

Time scaling is represented as:

$$x[2n]$$

This compresses the signal.

```
# Time scaling

n_scale = n[::2]
x_scale = x[::2]

# Plotting

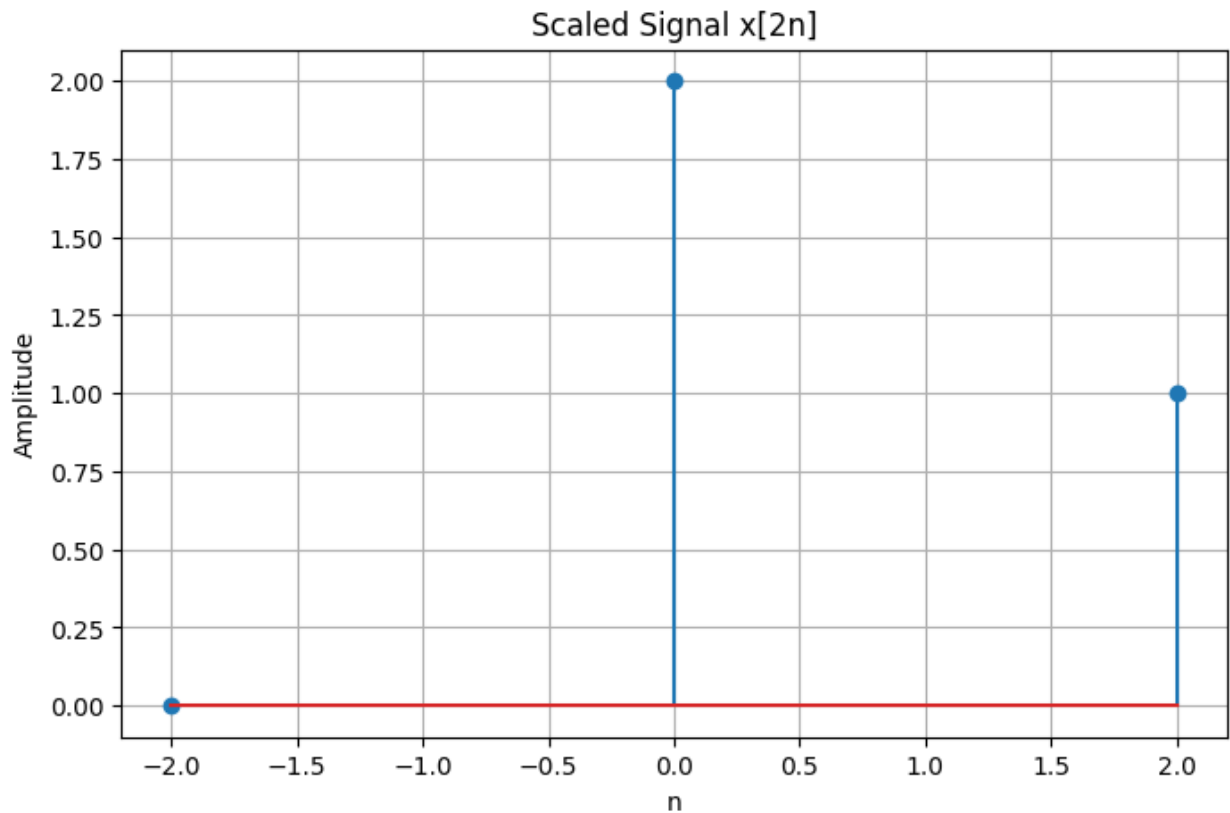
plt.figure(figsize=(8,5))

plt.stem(n_scale,x_scale)

plt.title("Scaled Signal x[2n]")
plt.xlabel("n")
plt.ylabel("Amplitude")

plt.grid()

plt.show()
```



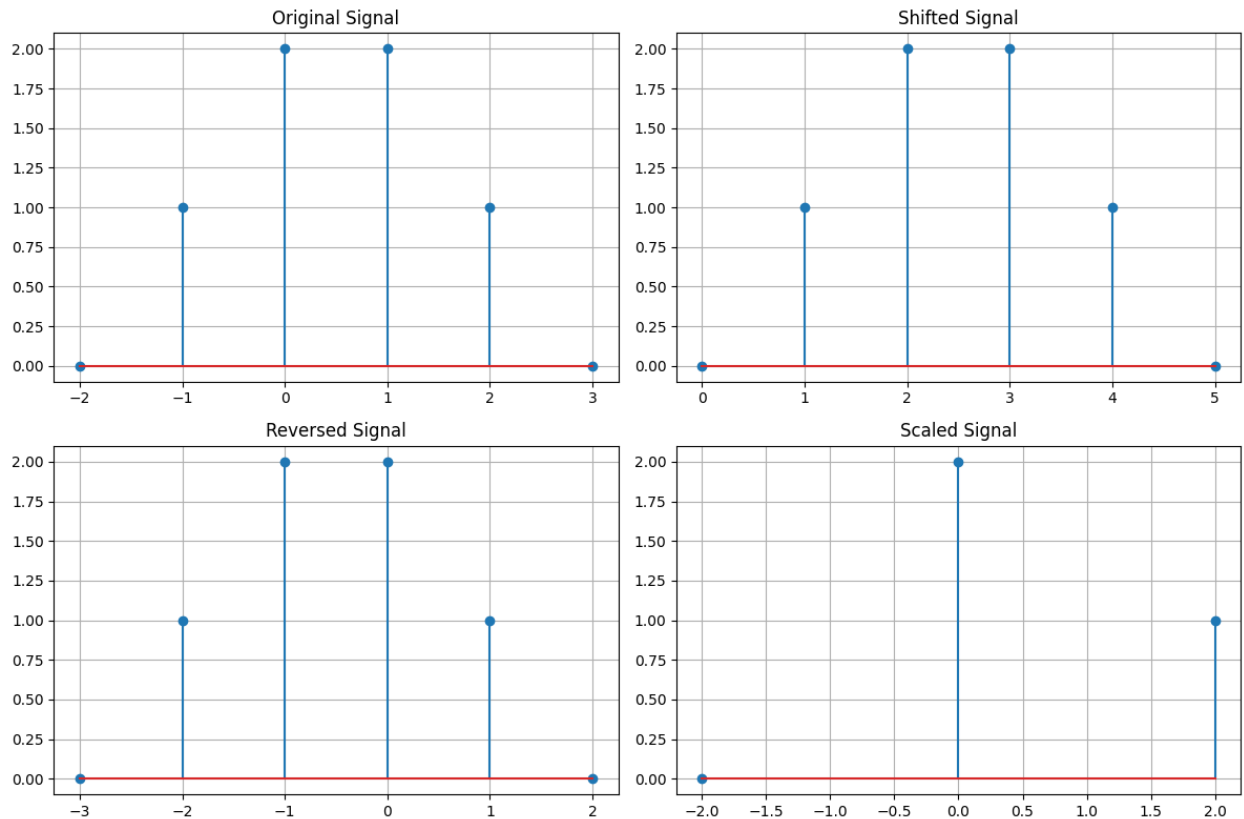
Observation

1. Time scaling compresses the signal.
2. Some samples are removed during compression.
3. The signal becomes denser in time.

Combined Visualization

```
plt.figure(figsize=(12,8))  
  
# Original signal  
plt.subplot(2,2,1)  
plt.stem(n,x)  
plt.title("Original Signal")  
plt.grid()  
  
# Shifted signal
```

```
plt.subplot(2,2,2)
plt.stem(n_shift,x)
plt.title("Shifted Signal")
plt.grid()
# Reversed signal
plt.subplot(2,2,3)
plt.stem(n_reverse,x)
plt.title("Reversed Signal")
plt.grid()
# Scaled signal
plt.subplot(2,2,4)
plt.stem(n_scale,x_scale)
plt.title("Scaled Signal")
plt.grid()
plt.tight_layout()
plt.show()
```



Comparison of Transformations

Transformation	Mathematical Form	Effect
Time Shift	$x[n-k]$	Delays signal
Time Reversal	$x[-n]$	Flips signal
Time Scaling	$x[2n]$	Compresses signal

#Periodic, Even and Odd Signals

Periodic Signal

A signal is periodic if:

$$x(t) = x(t+T)$$

where:

- T = fundamental period

Even Signal

A signal is even if:

$$x(t) = x(-t)$$

Even signals are symmetric about the vertical axis.

Odd Signal

A signal is odd if:

$$x(t) = -x(-t)$$

Odd signals possess origin symmetry.

Even-Odd Decomposition

Any signal can be represented as:

Even Part

$$x_e(t) = \frac{x(t) + x(-t)}{2}$$

Odd Part

$$x_o(t) = \frac{x(t) - x(-t)}{2}$$

Aim- 4

1. Generate periodic signals.
2. Generate even and odd signals.
3. Verify signal symmetry graphically.
4. Perform even-odd decomposition.
5. Visualize signal properties.

```
import numpy as np
import matplotlib.pyplot as plt
```

Example 1 — Periodic Signal

A sinusoidal signal is periodic.

Problem Statement

Generate and visualize a periodic sinusoidal signal with:

- Amplitude = 1
- Frequency = 2 Hz

```
# Parameters

A = 1
f = 2

# Time axis

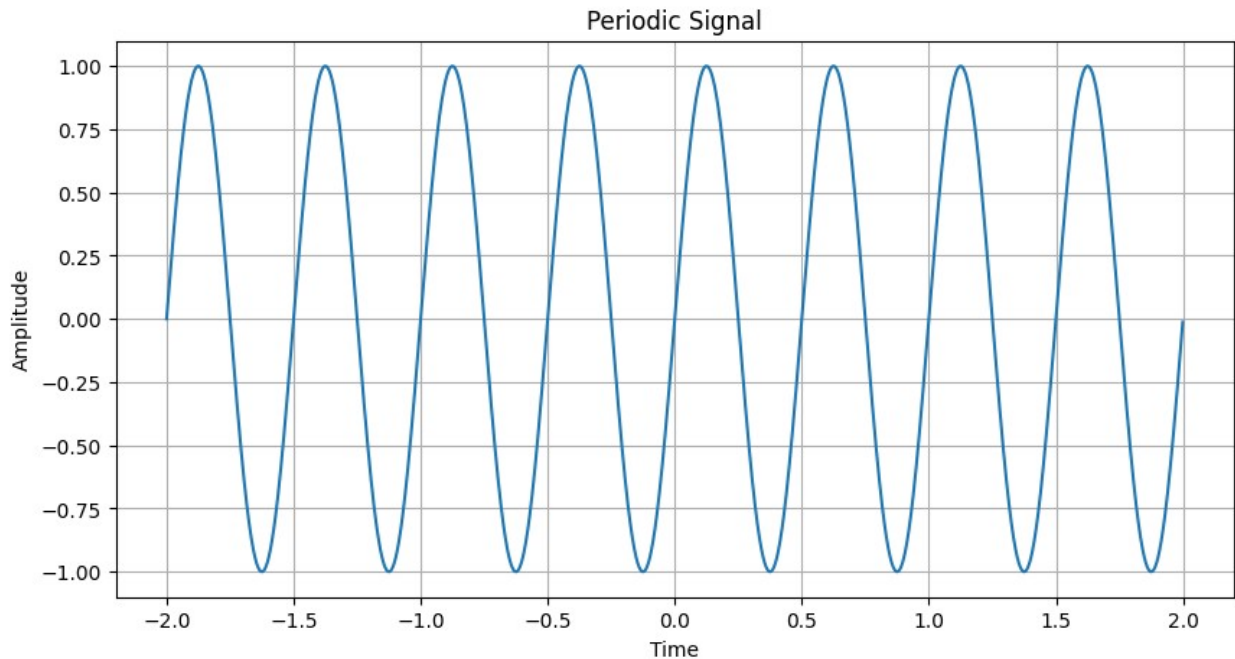
t = np.arange(-2,2,0.001)

# Periodic signal

x1 = A*np.sin(2*np.pi*f*t)

# Plotting

plt.figure(figsize=(10,5))
plt.plot(t,x1)
plt.title("Periodic Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```



Observation

1. The waveform repeats after equal intervals of time.
2. The signal is periodic in nature.
3. The fundamental period is:

$$T = \frac{1}{f} = \frac{1}{2} = 0.5 \text{ seconds}$$

Example 2 — Even Signal

Problem Statement

Generate and visualize an even signal.

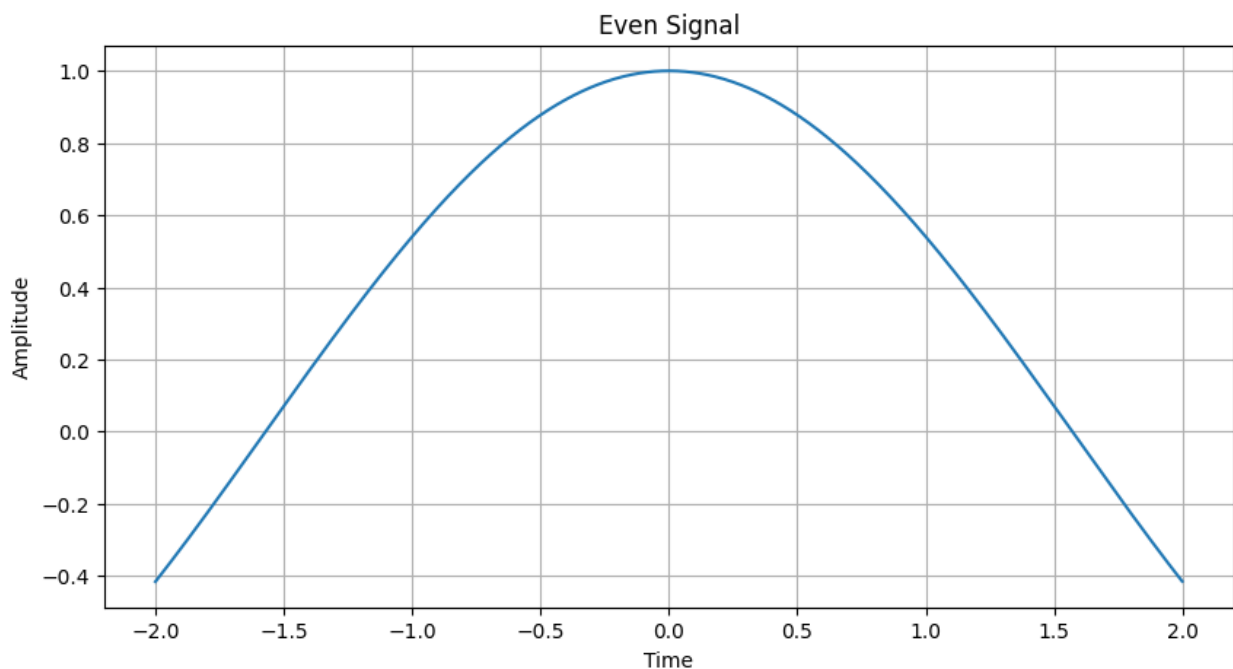
The cosine signal is an example of an even signal because:

$$\cos(t) = \cos(-t)$$

```
# Even signal
x_even = np.cos(t)

# Plotting
plt.figure(figsize=(10,5))
```

```
plt.plot(t,x_even)
plt.title("Even Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```



Observation

1. The signal is symmetric about the vertical axis.
2. Left and right halves of the signal are mirror images.
3. Hence, the signal is even.

Example 3 — Odd Signal

Problem Statement

Generate and visualize an odd signal.

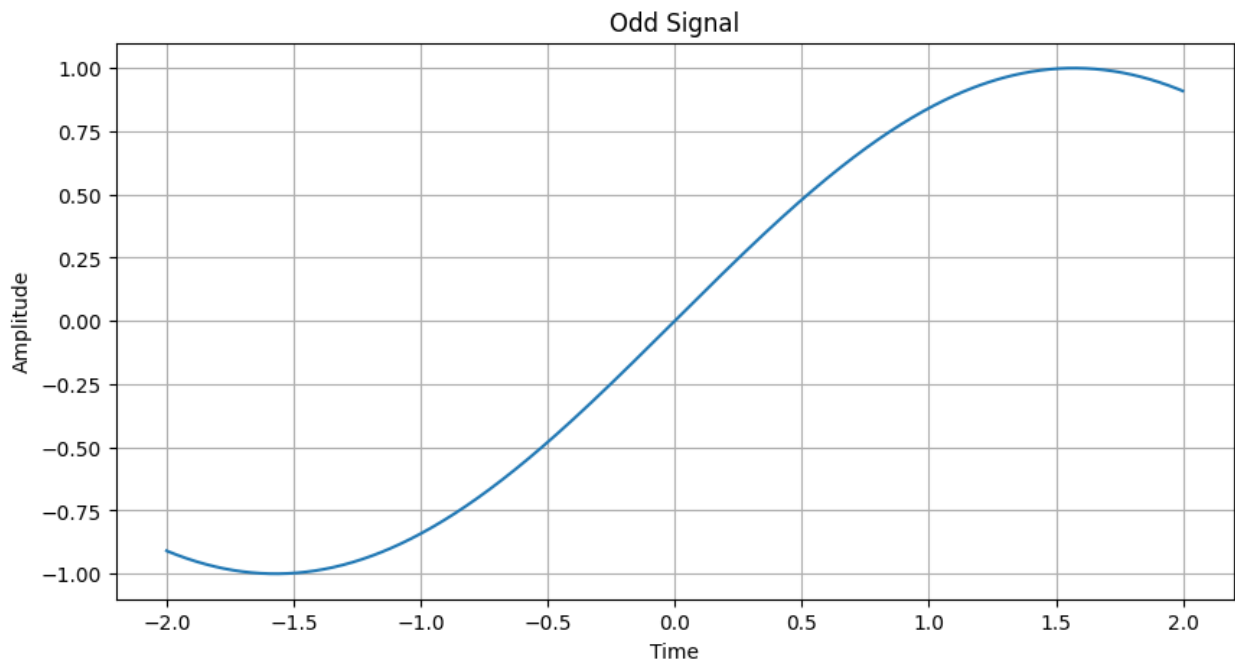
The sine signal is an example of an odd signal because:

$$\sin(t) = -\sin(-t)$$

```
# Odd signal
x_odd = np.sin(t)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,x_odd)
plt.title("Odd Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()

plt.show()
```



Observation

1. The signal possesses origin symmetry.
2. Positive and negative portions are symmetric about the origin.
3. Hence, the signal is odd.

Example 4 — Even-Odd Decomposition

Problem Statement

Decompose an arbitrary signal into even and odd components.

```
# Arbitrary signal
x = np.exp(-t)

# Flipped signal
x_flip = np.exp(t)

# Even part
x_e = (x + x_flip)/2

# Odd part
x_o = (x - x_flip)/2
```

Visualization of Even and Odd Components

```
plt.figure(figsize=(12,8))

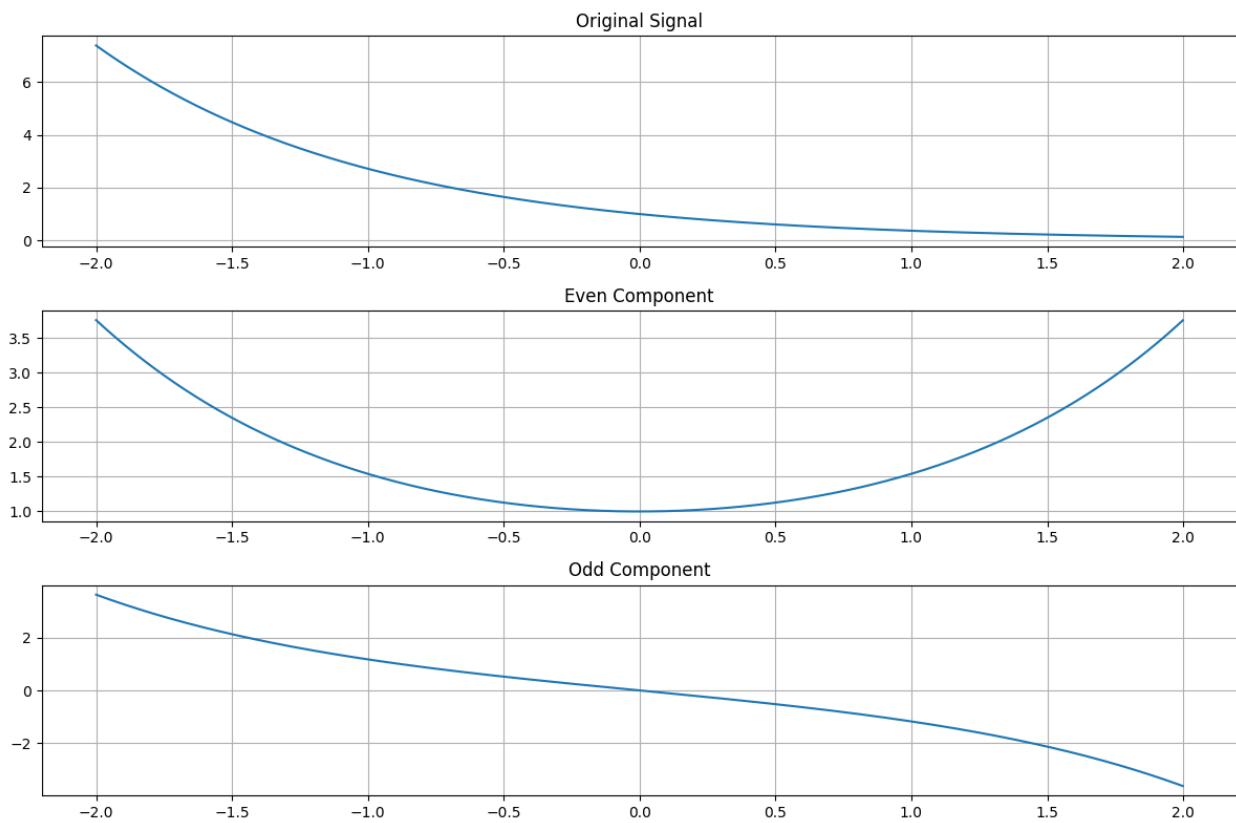
# Original signal
plt.subplot(3,1,1)
plt.plot(t,x)
plt.title("Original Signal")
plt.grid()

# Even component
plt.subplot(3,1,2)
plt.plot(t,x_e)
plt.title("Even Component")
plt.grid()

# Odd component
```

```
plt.subplot(3,1,3)
plt.plot(t,x_o)
plt.title("Odd Component")
plt.grid()
plt.tight_layout()

plt.show()
```

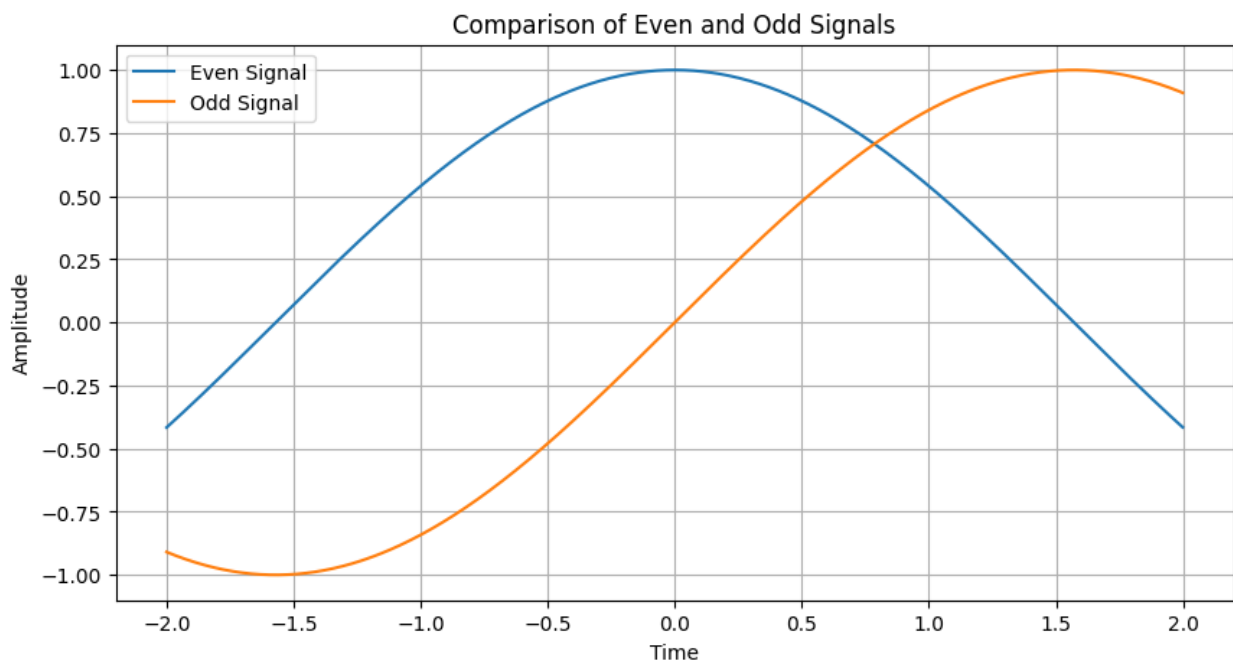


Comparison Table

Property	Even Signal	Odd Signal
Symmetry	Vertical axis	Origin
Mathematical Condition	$x(t)=x(-t)$	$x(t)=-x(-t)$
Example	$\cos(t)$	$\sin(t)$

Overlay Visualization of Even and Odd Signals

```
plt.figure(figsize=(10,5))  
plt.plot(t,x_even,label="Even Signal")  
plt.plot(t,x_odd,label="Odd Signal")  
plt.title("Comparison of Even and Odd Signals")  
plt.xlabel("Time")  
plt.ylabel("Amplitude")  
plt.legend()  
plt.grid()  
plt.show()
```



#Continuous-Time and Discrete-Time Complex Exponential and Sinusoidal Signals

A continuous-time complex exponential signal is represented as:

$$x(t) = A e^{j\omega t}$$

Using Euler's Formula:

$$e^{j\omega t} = \cos(\omega t) + j\sin(\omega t)$$

where:

- A = amplitude
 - ω = angular frequency
-

A discrete-time complex exponential signal is represented as:

$$x[n] = Ae^{j\Omega n}$$

where:

- Ω = discrete angular frequency
- n = discrete-time index

Aim- 5

1. Generate continuous-time sinusoidal signals.
2. Generate complex exponential signals.
3. Visualize real and imaginary parts.
4. Generate discrete-time sinusoidal signals.
5. Interpret complex signal behavior graphically.

```
import numpy as np
import matplotlib.pyplot as plt
```

Example 1 — Continuous-Time Sinusoidal Signal

Problem Statement

Generate a continuous-time sinusoidal signal with:

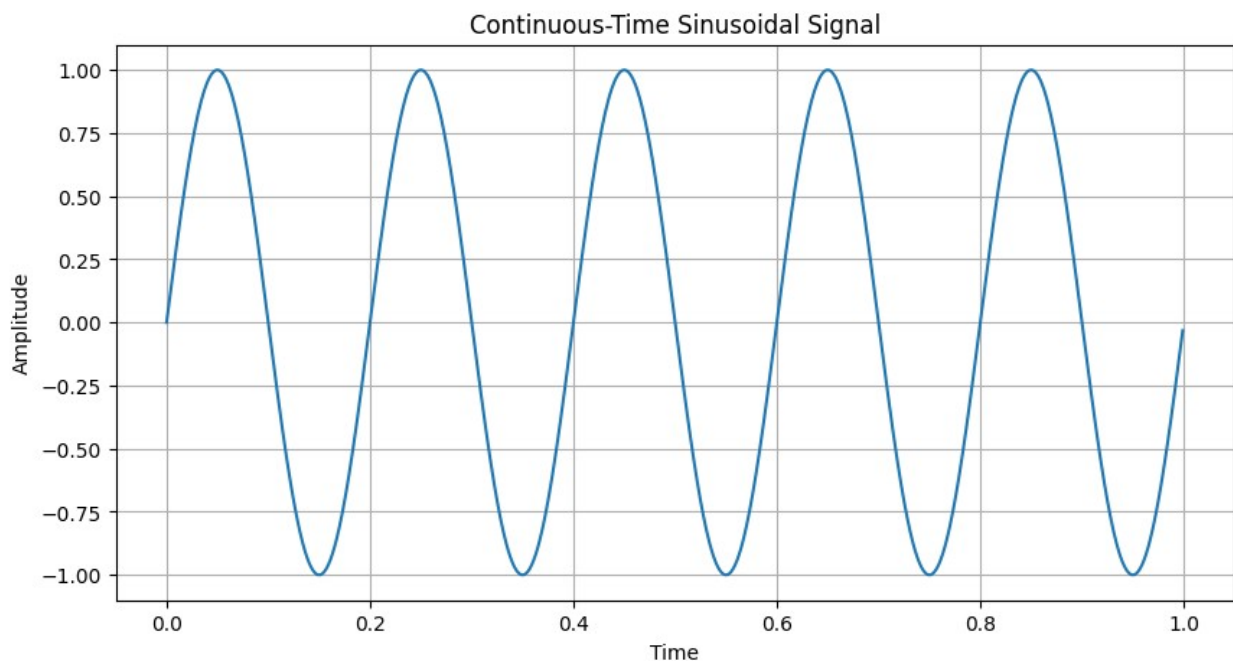
- Amplitude = 1
- Frequency = 5 Hz

```
# Parameters

A = 1
f = 5

# Angular frequency
```

```
omega = 2*np.pi*f
# Time axis
t = np.arange(0,1,0.001)
# Signal generation
x = A*np.sin(omega*t)
# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,x)
plt.title("Continuous-Time Sinusoidal Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```



Observation

1. The signal oscillates continuously with time.

2. The signal completes 5 cycles within 1 second.
3. The signal amplitude varies between +1 and -1.

Example 2 — Continuous-Time Complex Exponential Signal

Problem Statement

Generate the complex exponential signal:

$$x(t) = e^{j\omega t}$$

and visualize:

- real part
- imaginary part

```
# Complex exponential signal
x_complex = np.exp(1j*omega*t)

# Real and imaginary parts
x_real = np.real(x_complex)
x_imag = np.imag(x_complex)

# Plotting
plt.figure(figsize=(12,6))

# Real part
plt.subplot(2,1,1)
plt.plot(t,x_real)
plt.title("Real Part of Complex Exponential Signal")
plt.ylabel("Amplitude")
plt.grid()

# Imaginary part
plt.subplot(2,1,2)
plt.plot(t,x_imag)
```

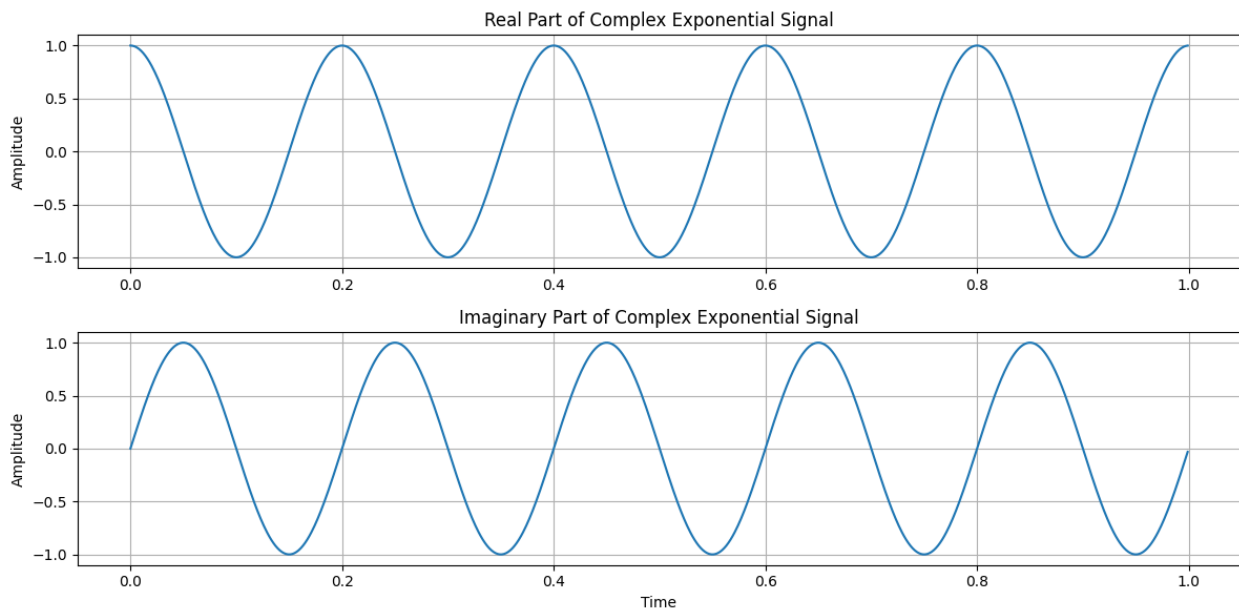
```
plt.title("Imaginary Part of Complex Exponential Signal")

plt.xlabel("Time")
plt.ylabel("Amplitude")

plt.grid()

plt.tight_layout()

plt.show()
```



Observation

1. The real part corresponds to a cosine signal.
2. The imaginary part corresponds to a sine signal.
3. Complex exponentials are composed of sinusoidal components.

Example 3 — Verification of Euler's Formula

Euler's Formula states:

$$e^{j\omega t} = \cos(\omega t) + j\sin(\omega t)$$

```
# Euler representation
lhs = np.exp(1j*omega*t)
```

```

rhs = np.cos(omega*t) + 1j*np.sin(omega*t)
# Difference
difference = np.max(np.abs(lhs-rhs))
print("Maximum difference =",difference)
Maximum difference = 0.0

```

Observation

The difference is nearly zero.

Hence, Euler's Formula is verified numerically.

Example 4 — Discrete-Time Sinusoidal Signal

Problem Statement

Generate a discrete-time sinusoidal signal using:

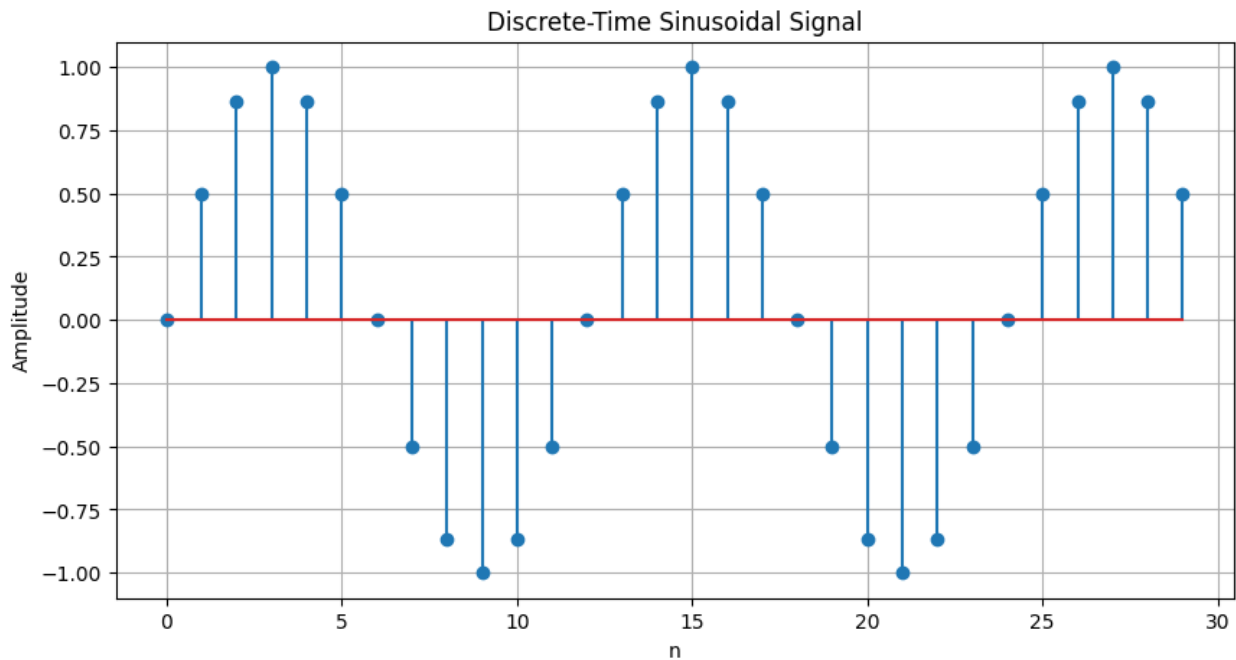
$$x[n] = \sin(\Omega n)$$

```

# Discrete-time index
n = np.arange(0,30)
# Discrete angular frequency
Omega = np.pi/6
# Discrete-time sinusoid
x_dt = np.sin(Omega*n)
# Plotting
plt.figure(figsize=(10,5))
plt.stem(n,x_dt)
plt.title("Discrete-Time Sinusoidal Signal")
plt.xlabel("n")
plt.ylabel("Amplitude")

```

```
plt.grid()
plt.show()
```



Observation

1. The signal exists only at discrete values of n .
2. Stem plots are used for DT signals.
3. The DT sinusoid appears sampled in nature.

Example 5 — Discrete-Time Complex Exponential Signal

```
# DT complex exponential signal
x_dt_complex = np.exp(1j*0omega*n)

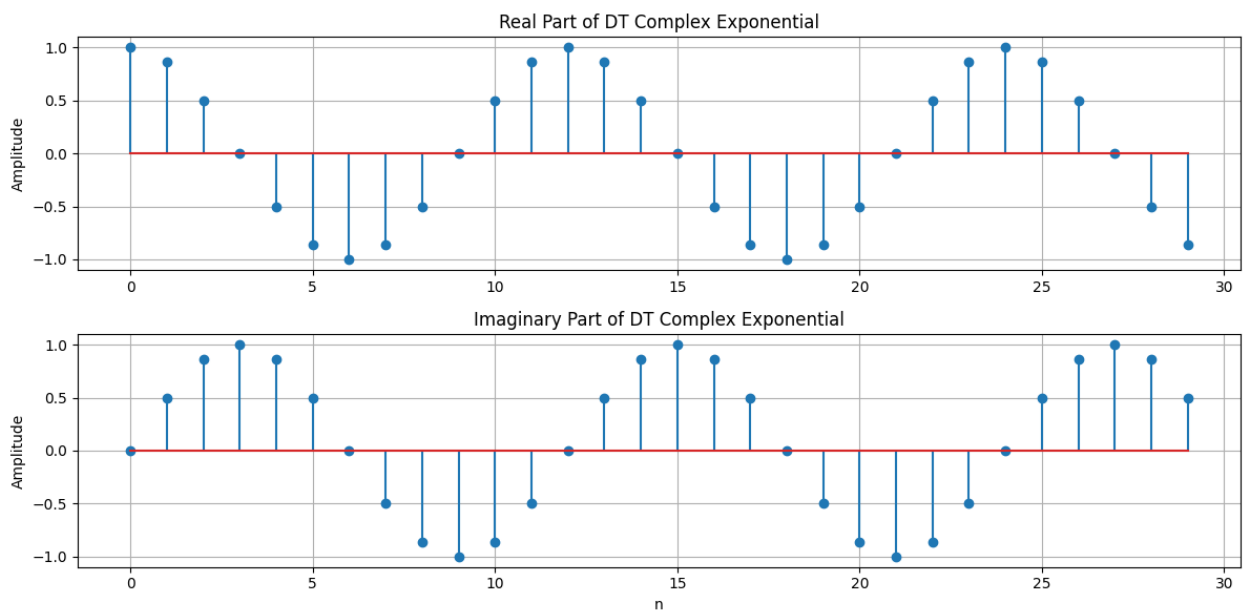
# Real and imaginary parts
x_dt_real = np.real(x_dt_complex)
x_dt_imag = np.imag(x_dt_complex)

# Plotting
```

```

plt.figure(figsize=(12,6))
# Real part
plt.subplot(2,1,1)
plt.stem(n,x_dt_real)
plt.title("Real Part of DT Complex Exponential")
plt.ylabel("Amplitude")
plt.grid()
# Imaginary part
plt.subplot(2,1,2)
plt.stem(n,x_dt_imag)
plt.title("Imaginary Part of DT Complex Exponential")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.grid()
plt.tight_layout()
plt.show()

```



Comparison Between CT and DT Signals

Continuous-Time Signal	Discrete-Time Signal
Defined for all t	Defined only for integer n
Uses line plots	Uses stem plots
Smooth waveform	Sampled waveform
Uses ω	Uses Ω

Magnitude and Phase of Complex Exponential

```
# Magnitude
magnitude = np.abs(x_complex)

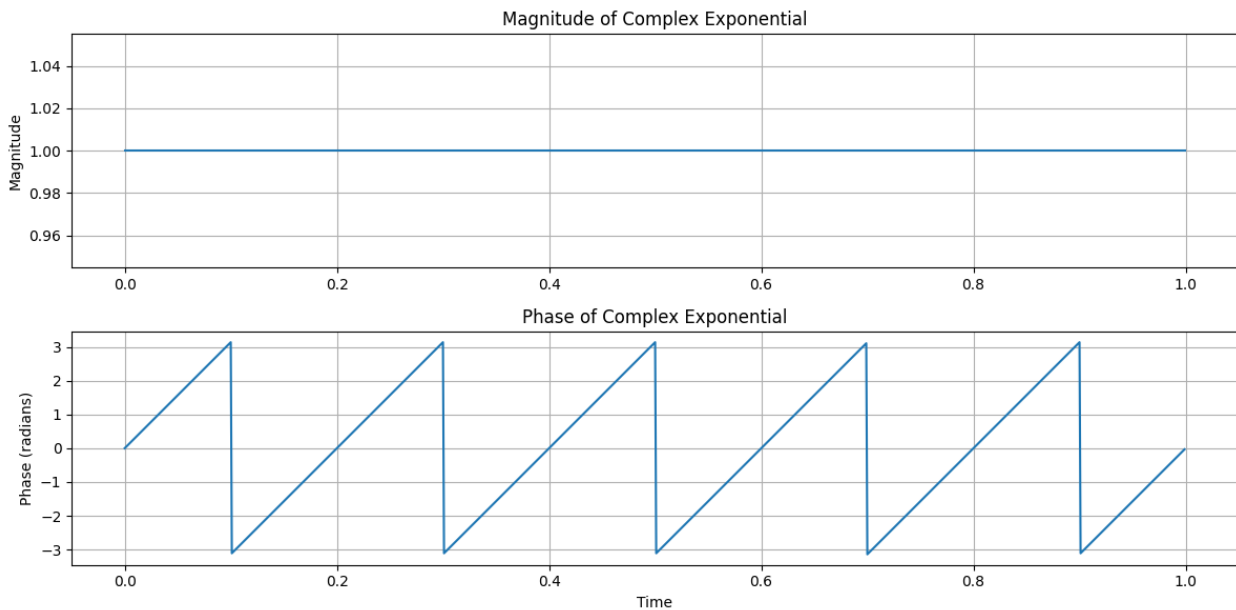
# Phase
phase = np.angle(x_complex)

# Plotting
plt.figure(figsize=(12,6))

# Magnitude
plt.subplot(2,1,1)
plt.plot(t,magnitude)
plt.title("Magnitude of Complex Exponential")
plt.ylabel("Magnitude")
plt.grid()

# Phase
plt.subplot(2,1,2)
plt.plot(t,phase)
plt.title("Phase of Complex Exponential")
plt.xlabel("Time")
plt.ylabel("Phase (radians)")
plt.grid()
plt.tight_layout()
```

```
plt.show()
```



#Unit Impulse and Unit Step Signals

Unit Step Signal

The unit step signal is represented as:

$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}$$

The unit step signal is used to represent switching operations.

Unit Impulse Signal

The unit impulse signal is represented as:

$$\delta(t)$$

The impulse signal has:

- infinite amplitude
 - zero width
 - unit area
-

Relationship Between Impulse and Step Signals

$$\delta(t) = \frac{d}{dt} u(t)$$

Aim- 6

1. Generate unit step signals.
2. Generate delayed step signals.
3. Approximate unit impulse signals.
4. Construct rectangular pulses.
5. Visualize impulse-step relationships.

```
import numpy as np
import matplotlib.pyplot as plt
```

Example 1 — Unit Step Signal

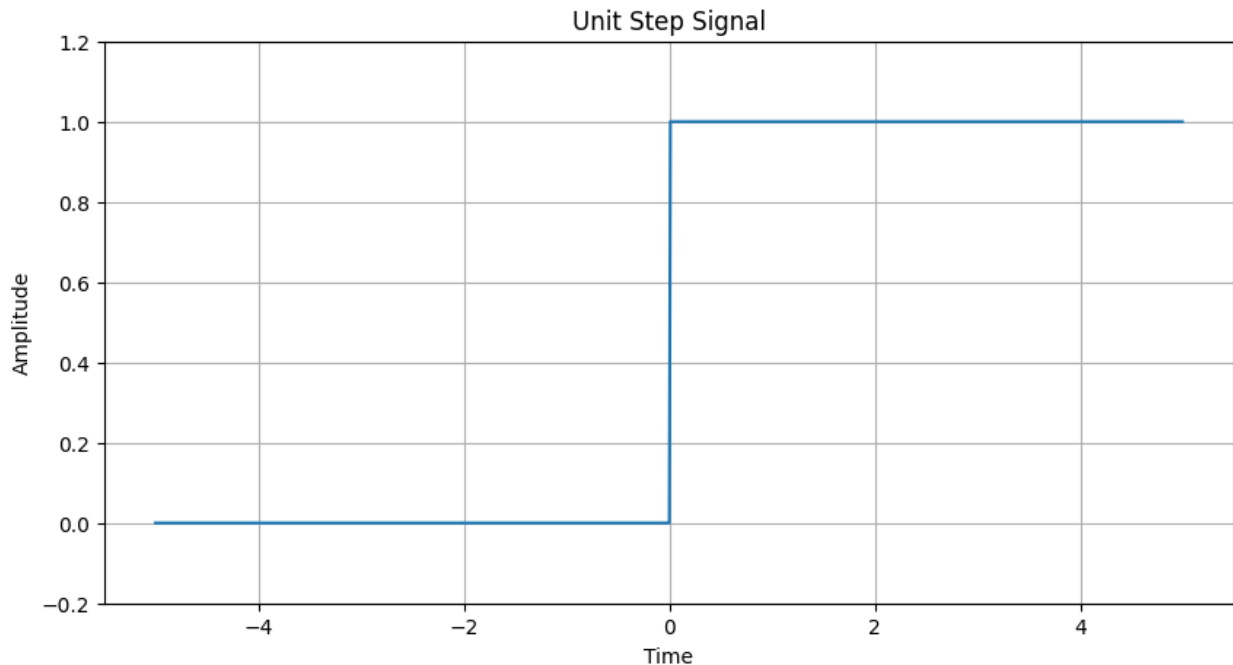
Problem Statement

Generate and visualize the unit step signal.

```
# Time axis
t = np.arange(-5,5,0.01)

# Unit step signal
u = np.where(t >= 0,1,0)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,u)
plt.title("Unit Step Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.ylim(-0.2,1.2)
plt.grid()
plt.show()
```



Example 2 — Delayed Unit Step Signal

Problem Statement

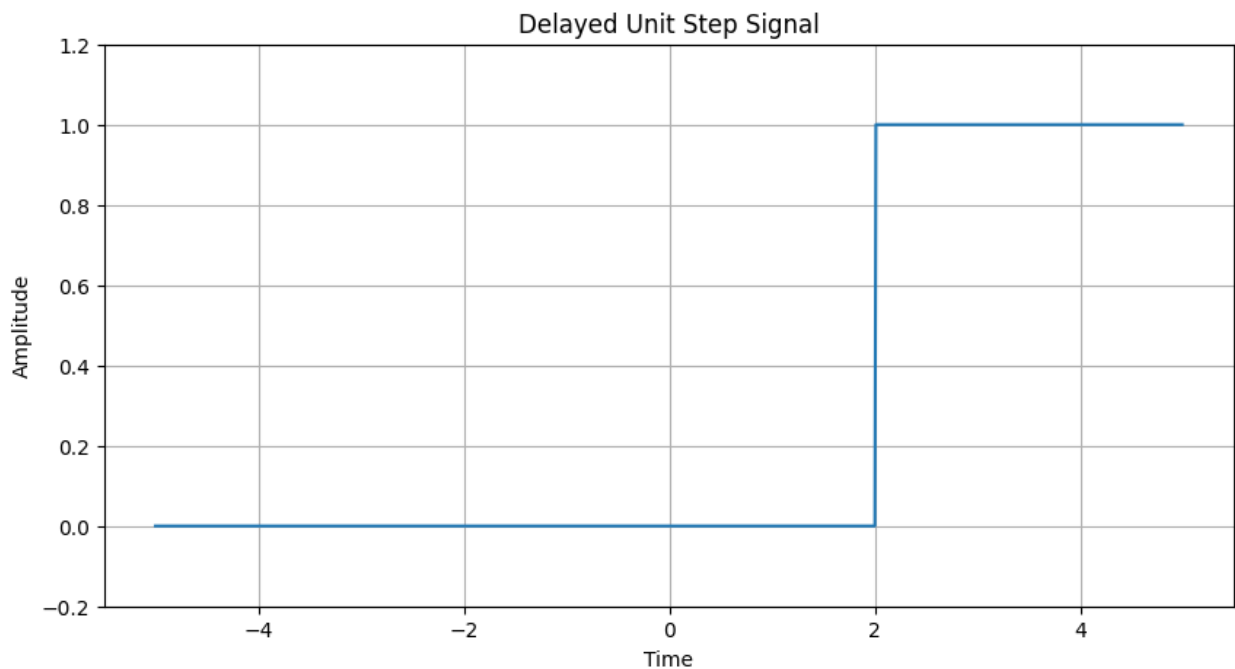
Generate a delayed unit step signal:

$$u(t-2)$$

```
# Delayed unit step
u_delay = np.where(t >= 2,1,0)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,u_delay)
plt.title("Delayed Unit Step Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.ylim(-0.2,1.2)
plt.grid()
```

```
plt.show()
```



Example 3 — Unit Impulse Approximation

Problem Statement

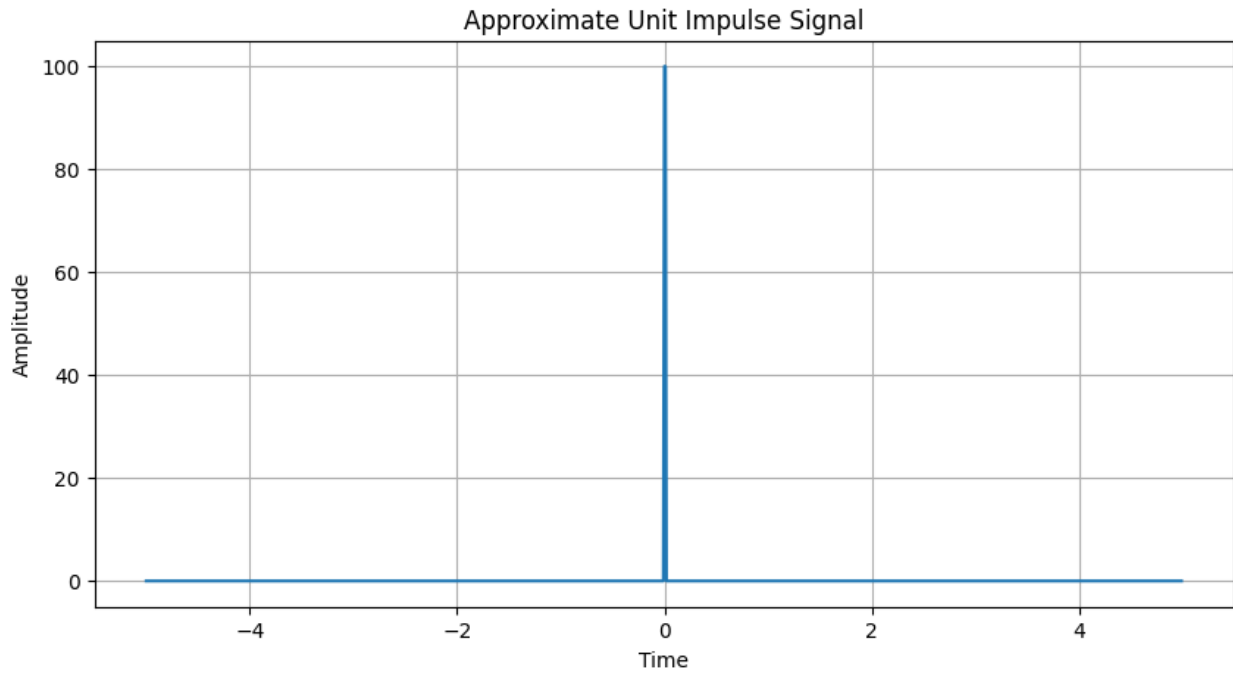
Approximate the unit impulse signal using a narrow pulse.

```
# Impulse approximation
impulse = np.zeros(len(t))

# Approximate impulse location
impulse[np.abs(t) < 0.01] = 100

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,impulse)
plt.title("Approximate Unit Impulse Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
```

```
plt.grid()  
plt.show()
```



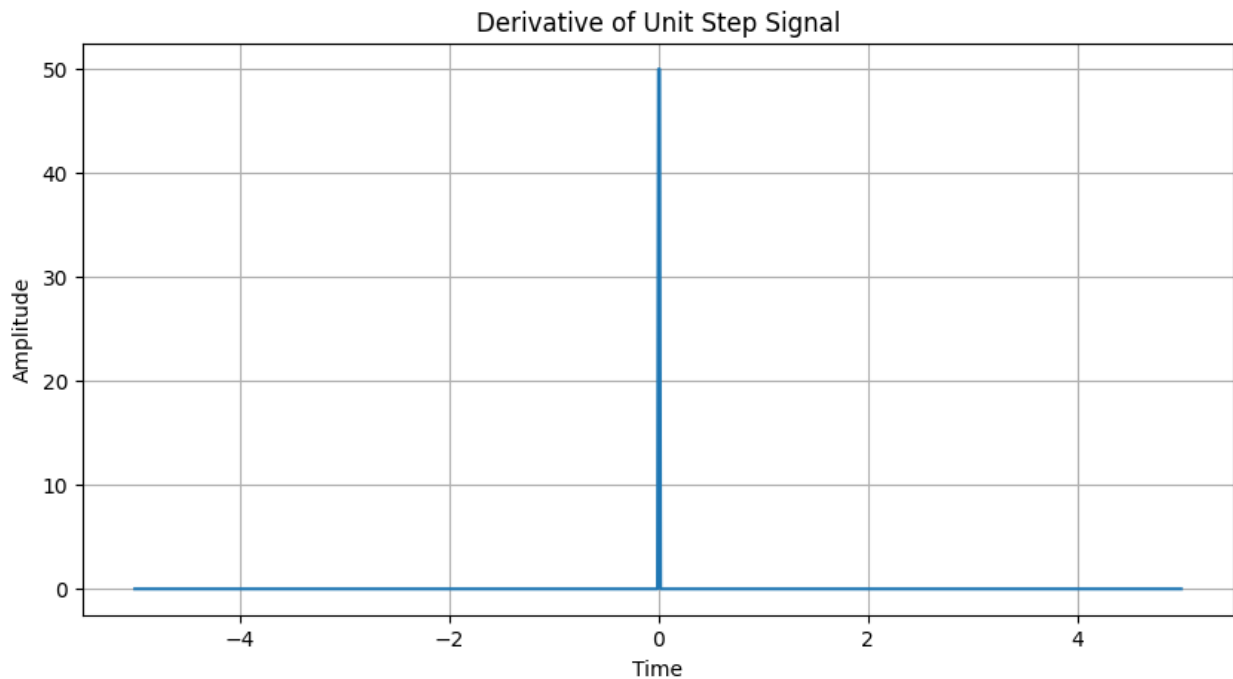
Example 4 — Relationship Between Step and Impulse Signals

The impulse signal is the derivative of the step signal:

$$\delta(t) = \frac{d}{dt} u(t)$$

```
# Numerical derivative of unit step  
du = np.gradient(u,t)  
  
# Plotting  
plt.figure(figsize=(10,5))  
plt.plot(t,du)  
plt.title("Derivative of Unit Step Signal")  
plt.xlabel("Time")
```

```
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```



Example 5 — Rectangular Pulse Using Step Signals

Problem Statement

Generate a rectangular pulse using step functions.

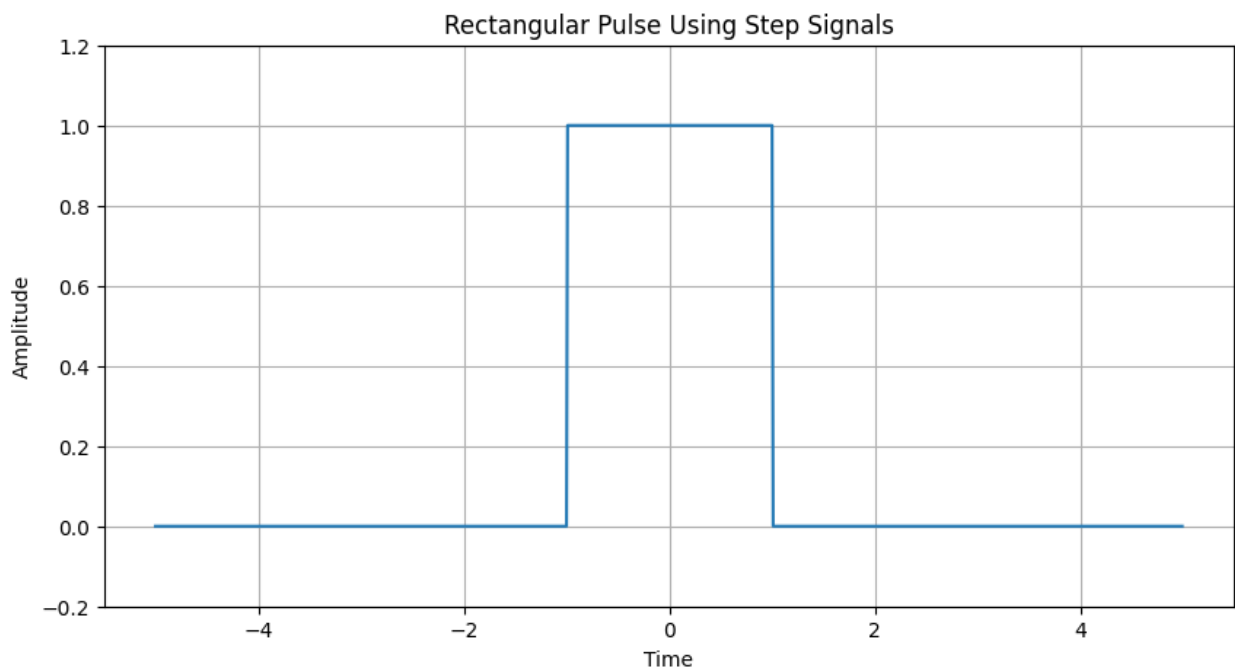
$$x(t) = u(t+1) - u(t-1)$$

```
# Step functions
u1 = np.where(t >= -1, 1, 0)
u2 = np.where(t >= 1, 1, 0)
# Rectangular pulse
rect = u1 - u2
```

```

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,rect)
plt.title("Rectangular Pulse Using Step Signals")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.ylim(-0.2,1.2)
plt.grid()
plt.show()

```



Comparison Table

Signal	Description
Unit Step	Sudden switching signal
Delayed Step	Shifted switching signal
Impulse	Instantaneous spike
Rectangular Pulse	Finite-duration pulse

#Causality, Stability and Time Invariance

Important properties of systems include:

1. Causality
 2. Stability
 3. Time Invariance
-

Causal System

A system is causal if the output depends only on:

- present inputs
- past inputs

and NOT on future inputs.

Example:

$$y(t) = x(t - 1)$$

Stable System

A system is stable if every bounded input produces a bounded output.

Example:

$$|x(t)| < M \Rightarrow |y(t)| < N$$

Time Invariant System

A system is time invariant if:

- shifting the input
- shifts the output by the same amount

Example:

$$y(t) = x(t - 2)$$

Aim- 7

1. Understand causality.
2. Understand stability.

3. Understand time invariance.
4. Simulate input-output relationships.
5. Verify system properties graphically.

```
import numpy as np
import matplotlib.pyplot as plt
```

Example 1 — Causal System

Problem Statement

Consider the system:

$$y(t) = x(t - 1)$$

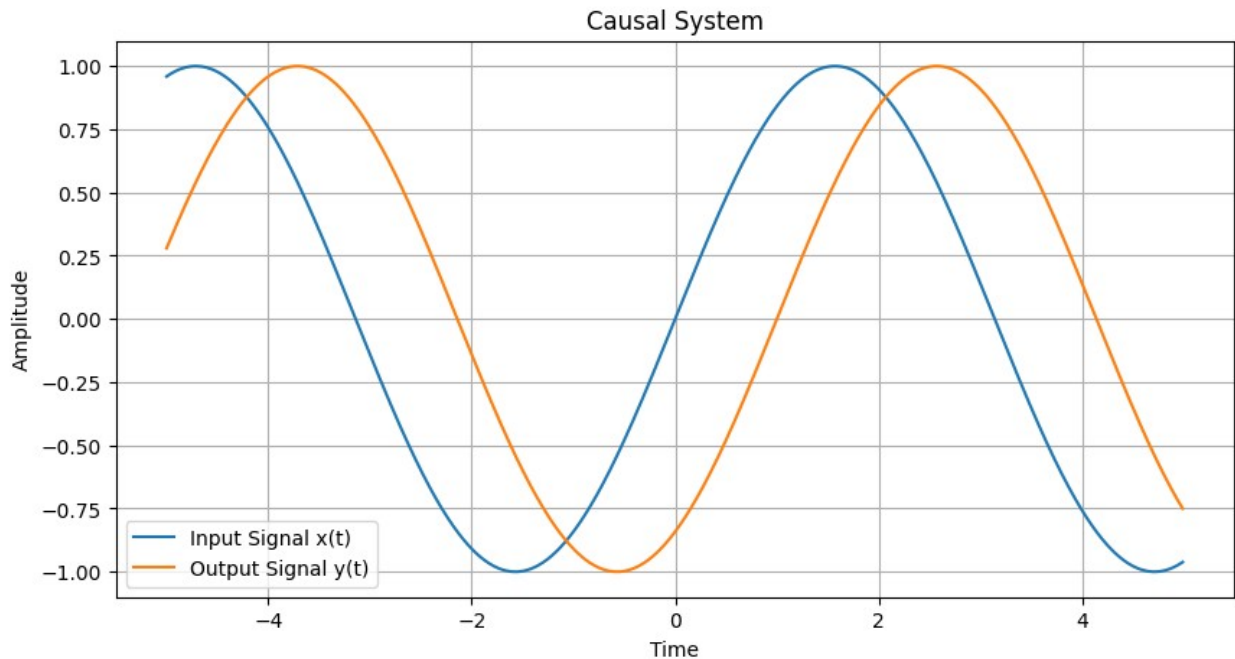
Generate input and output signals and verify causality.

```
# Time axis
t = np.arange(-5, 5, 0.01)

# Input signal
x = np.sin(t)

# Output signal
y = np.sin(t-1)

# Plotting
plt.figure(figsize=(10, 5))
plt.plot(t, x, label="Input Signal x(t)")
plt.plot(t, y, label="Output Signal y(t)")
plt.title("Causal System")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.show()
```



Observation

1. The output is a delayed version of the input.
2. The system depends only on past values.
3. Hence, the system is causal.

Example 2 — Non-Causal System

Problem Statement

Consider the system:

$$y(t) = x(t+1)$$

Generate input and output signals and analyze causality.

```
# Output signal
y_noncausal = np.sin(t+1)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Input Signal x(t)")
```

```
plt.plot(t,y_noncausal,label="Output Signal y(t)")

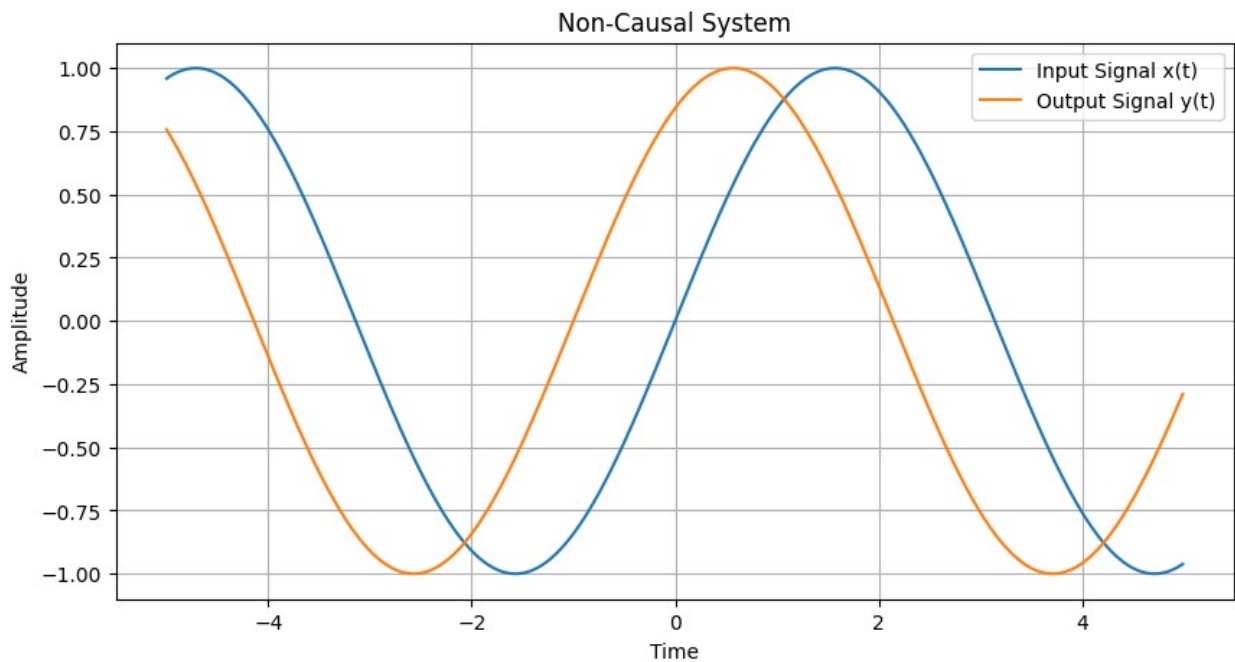
plt.title("Non-Causal System")

plt.xlabel("Time")
plt.ylabel("Amplitude")

plt.legend()

plt.grid()

plt.show()
```



Observation

1. The output depends on future values of the input.
2. Such systems are not physically realizable.
3. Hence, the system is non-causal.

Example 3 — Stable System

Problem Statement

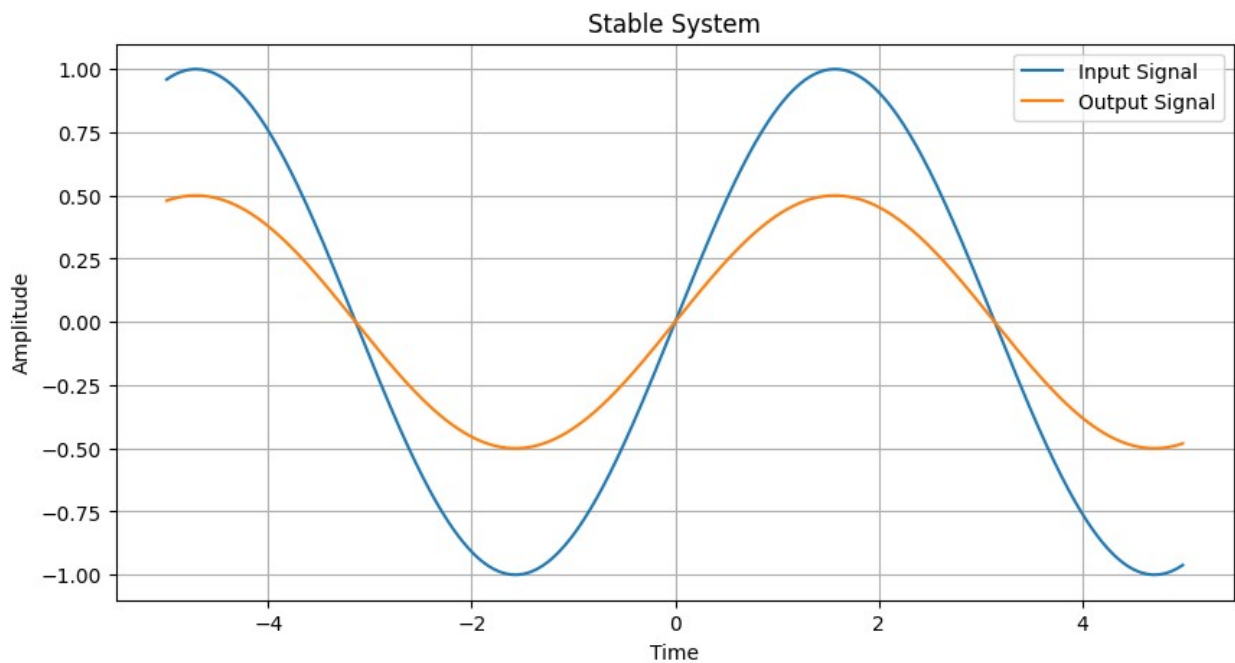
Consider the system:

$$y(t) = 0.5x(t)$$

Verify system stability.

```
# Stable output
y_stable = 0.5*x

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Input Signal")
plt.plot(t,y_stable,label="Output Signal")
plt.title("Stable System")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.show()
```



Observation

1. The input signal is bounded.
2. The output signal is also bounded.

3. Hence, the system is stable.

Example 4 — Unstable System

Problem Statement

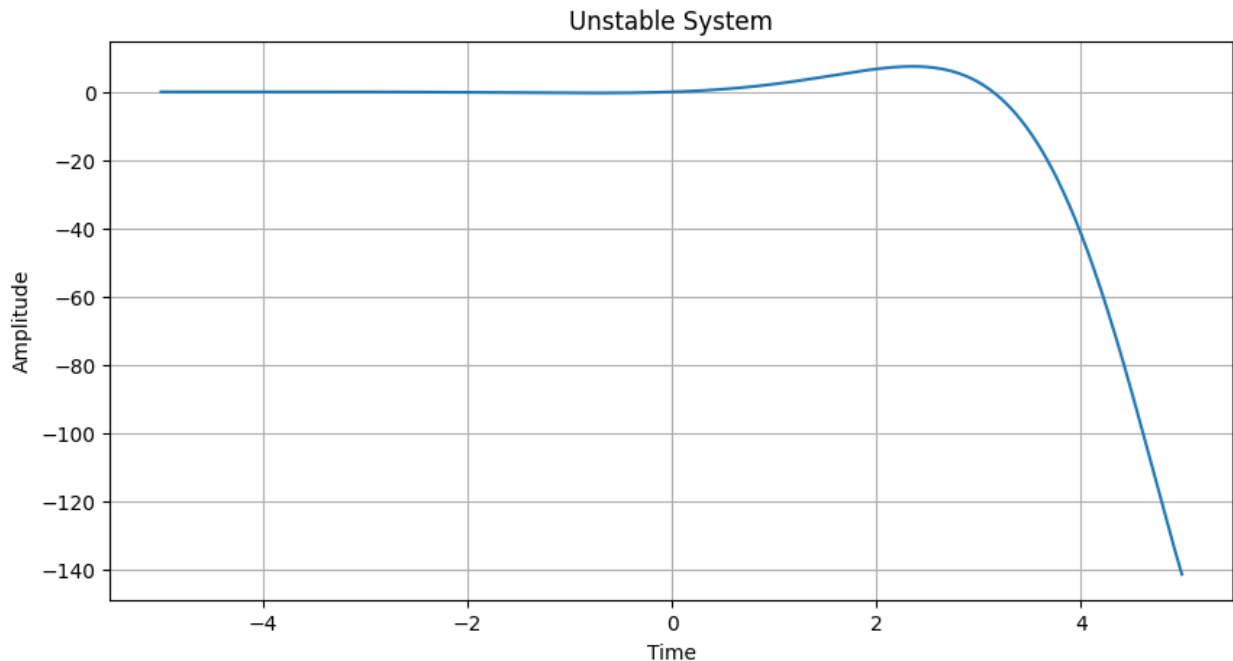
Consider the system:

$$y(t) = e^t x(t)$$

Analyze system stability.

```
# Unstable output
y_unstable = np.exp(t)*x

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,y_unstable)
plt.title("Unstable System")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```



Example 5 — Time Invariant System

Problem Statement

Consider the system:

$$y(t) = x(t - 2)$$

Verify time invariance graphically.

```
# Original output
y1 = np.sin(t-2)

# Shifted input
x_shift = np.sin((t-1))

# Shifted output
y2 = np.sin((t-1)-2)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,y1,label="Original Output")
```

```
plt.plot(t,y2,label="Shifted Output")

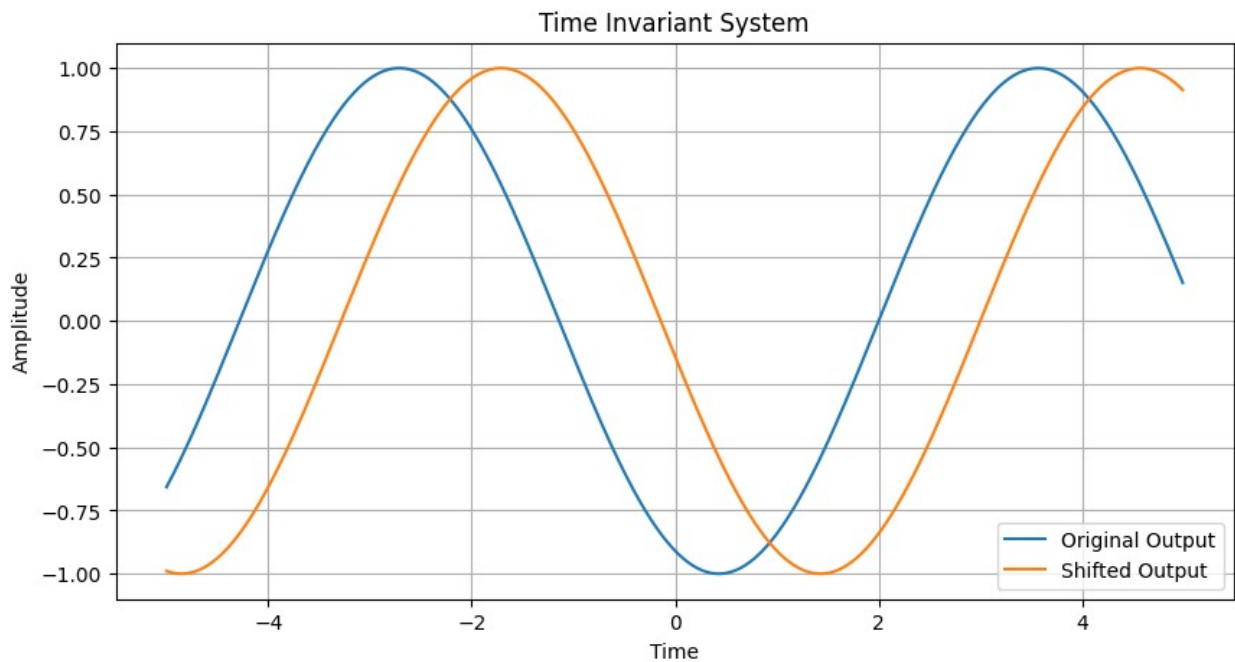
plt.title("Time Invariant System")

plt.xlabel("Time")
plt.ylabel("Amplitude")

plt.legend()

plt.grid()

plt.show()
```



Comparison Table

Property	Description
Causal	Depends only on present/past inputs
Non-Causal	Depends on future inputs
Stable	Bounded input gives bounded output
Unstable	Bounded input gives unbounded output
Time Invariant	Time shift preserved

#Basic Problems

Question 1

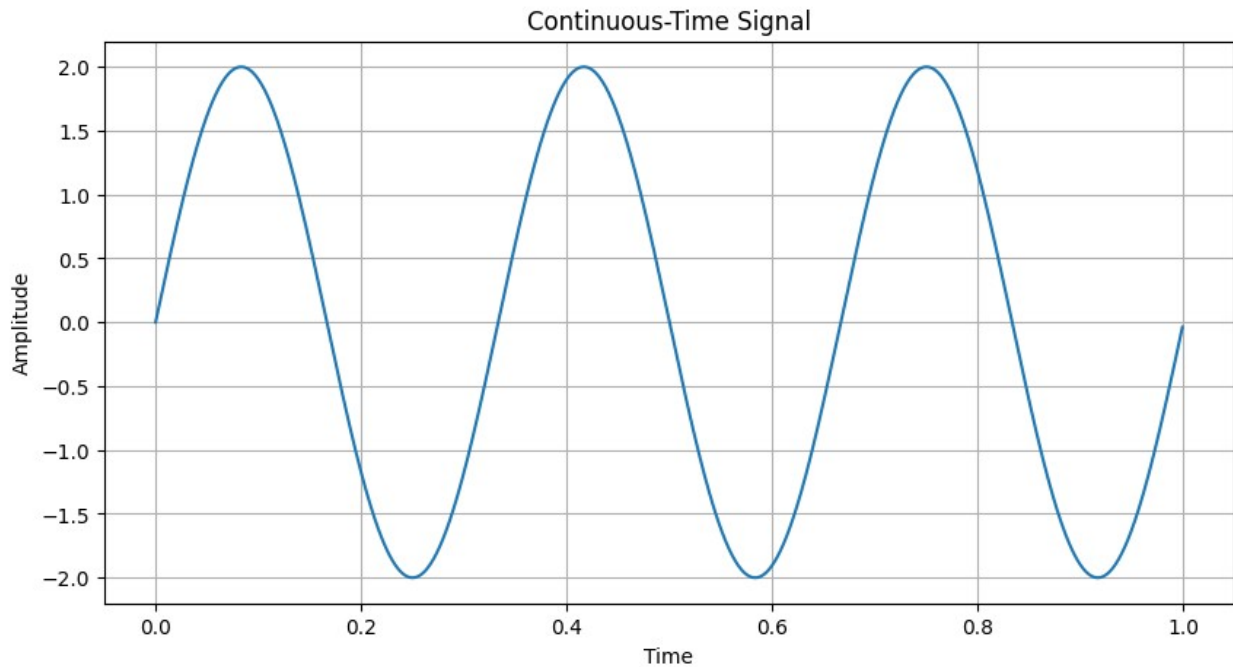
Generate the signal:

$$x(t) = 2 \sin(2\pi 3t)$$

```
# Time axis
t = np.arange(0,1,0.001)

# Signal generation
x1 = 2*np.sin(2*np.pi*3*t)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,x1)
plt.title("Continuous-Time Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```



Observations

1. The signal amplitude is 2.
2. The signal frequency is 3 Hz.
3. The waveform repeats periodically.
4. The signal is a continuous signal.

Question 2

Generate the DT signal:

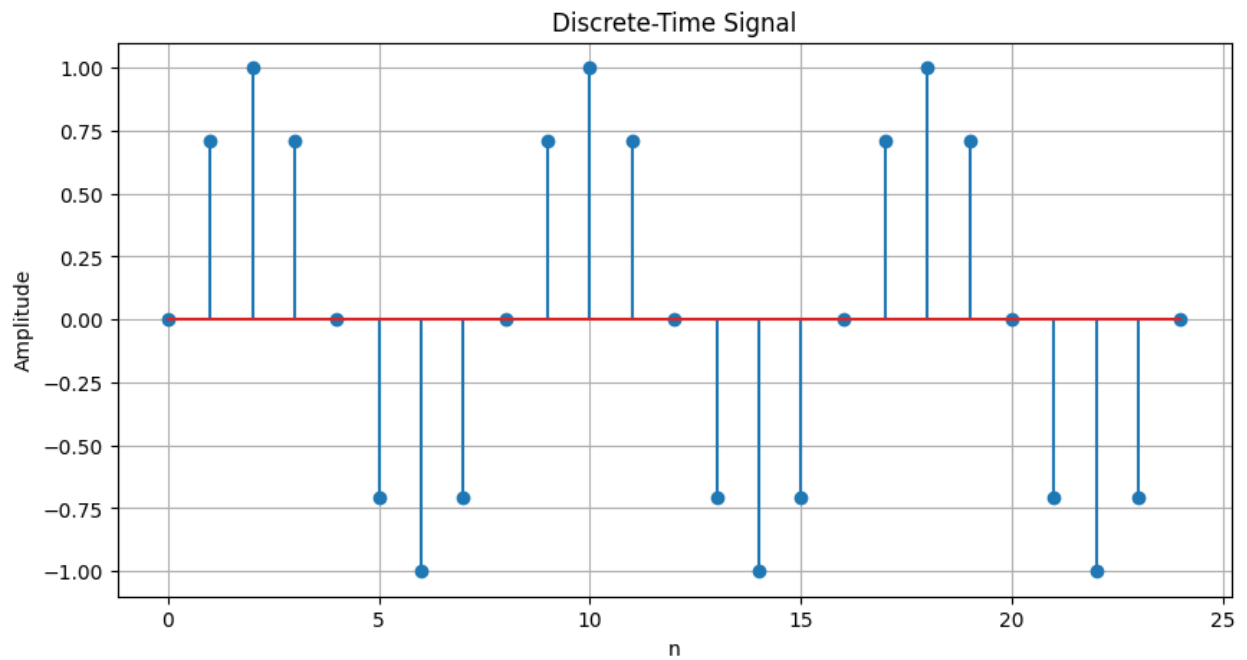
$$x[n] = \sin\left(\frac{\pi n}{4}\right)$$

```
# Discrete index
n = np.arange(0,25)

# DT signal
x2 = np.sin(np.pi*n/4)

# Plotting
plt.figure(figsize=(10,5))
plt.stem(n,x2)
```

```
plt.title("Discrete-Time Signal")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```



Question 3

Generate and visualize:

$$x(t-2)$$

for the signal:

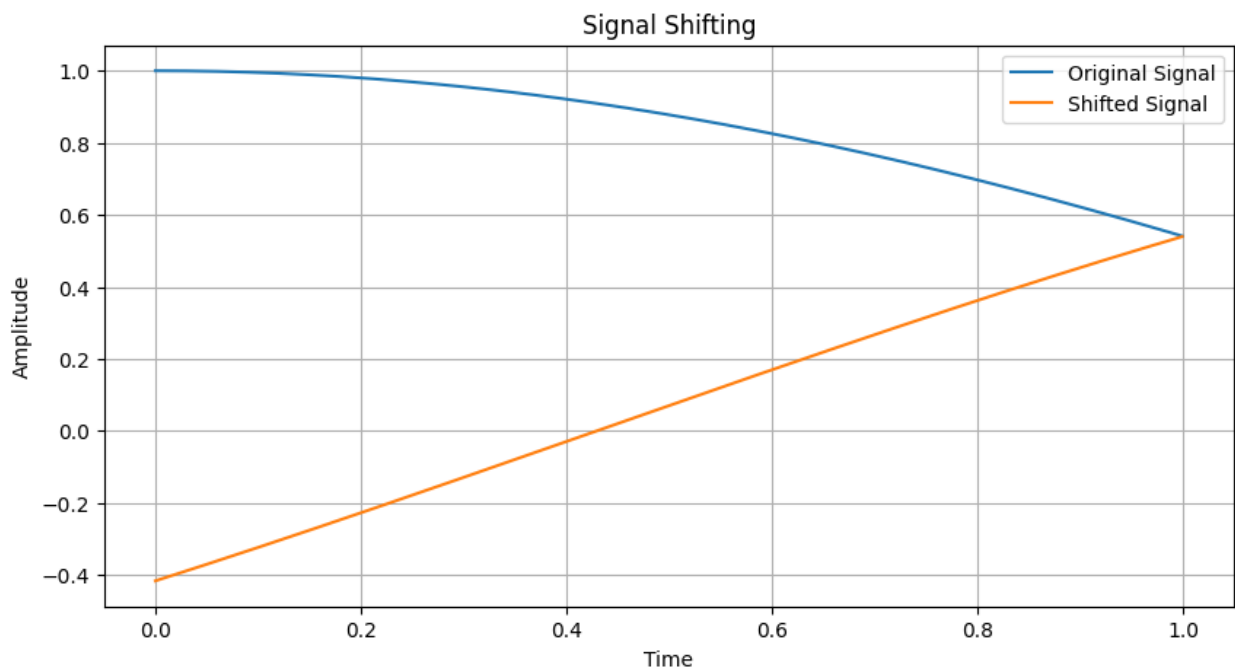
$$x(t) = \cos(t)$$

```
# Original signal
x_original = np.cos(t)
# Shifted signal
x_shift = np.cos(t-2)
```

```

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t,x_original,label="Original Signal")
plt.plot(t,x_shift,label="Shifted Signal")
plt.title("Signal Shifting")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.show()

```



Observation

1. The shifted signal moves right by 2 units.
2. Signal shape remains unchanged.
3. Time shifting changes signal position only.

Question 4

Compute the energy of:

$$x(t)=e^{-t}u(t)$$

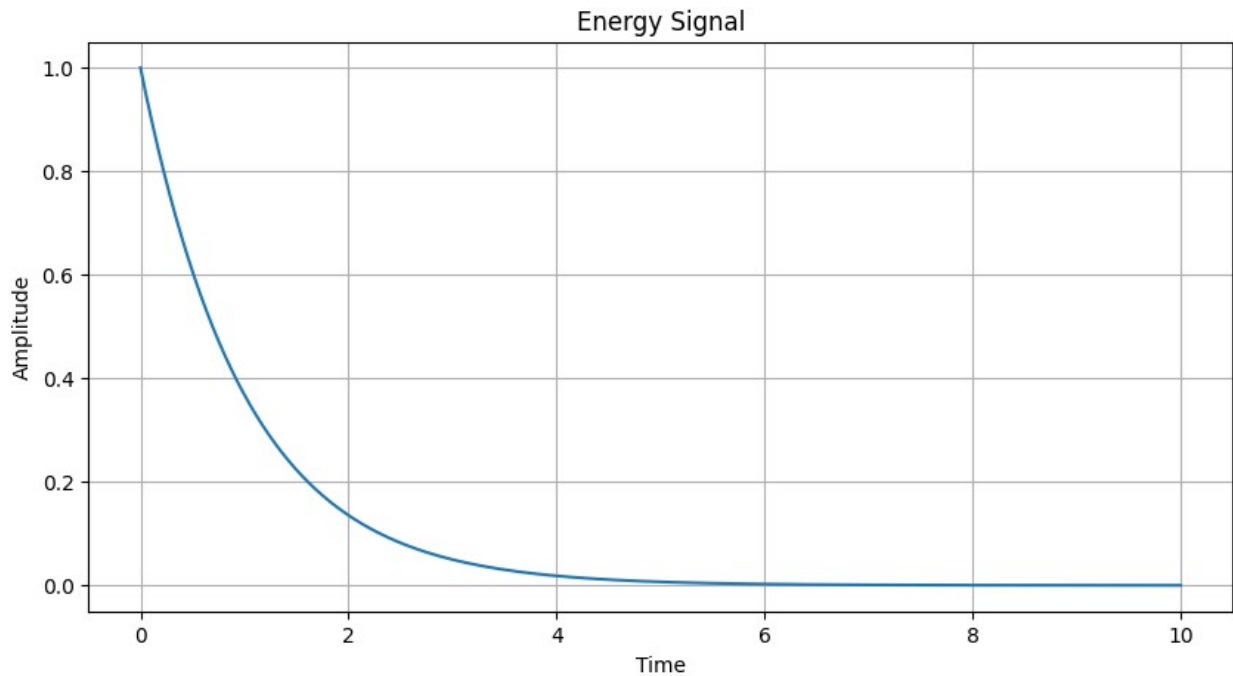
```
# Time axis
t2 = np.arange(0,10,0.001)

# Signal
x_energy = np.exp(-t2)

# Energy computation
E = np.trapezoid(np.abs(x_energy)**2,t2)
print("Energy of the signal =",E)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t2,x_energy)
plt.title("Energy Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
plt.show()

Energy of the signal = 0.5000001656340152
```



Observation

1. The signal decays with time.
2. The computed energy is finite.
3. Hence, the signal is an energy signal.

Question 5

Verify whether:

- cosine signal is even
- sine signal is odd

```
# Time axis
t3 = np.arange(-10,10,0.01)

# Signals
x_even = np.cos(t3)
x_odd = np.sin(t3)

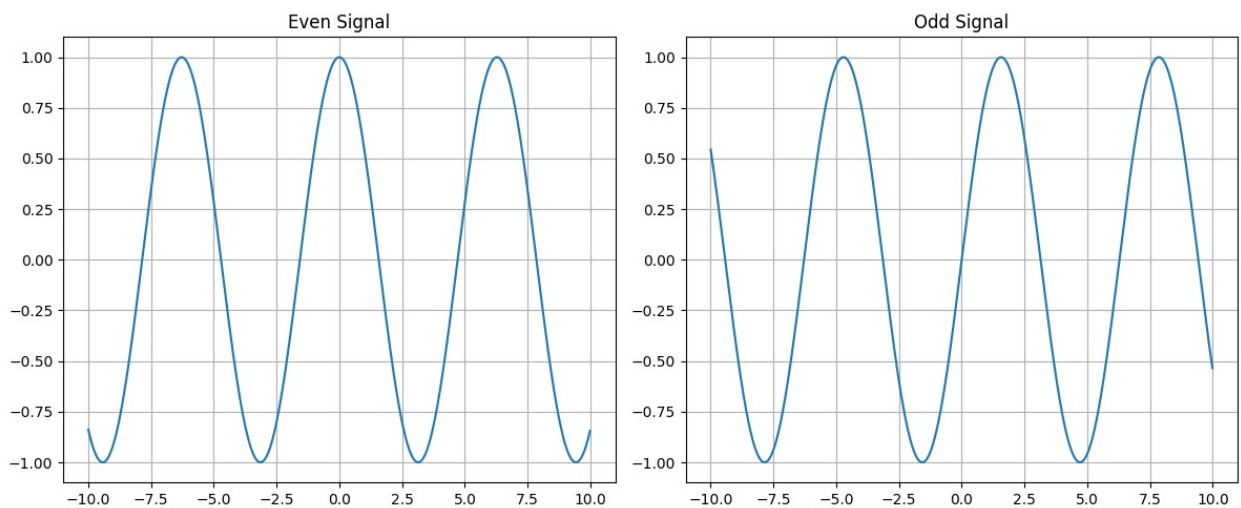
# Plotting
plt.figure(figsize=(12,5))

# Even signal
```

```

plt.subplot(1,2,1)
plt.plot(t3,x_even)
plt.title("Even Signal")
plt.grid()
# Odd signal
plt.subplot(1,2,2)
plt.plot(t3,x_odd)
plt.title("Odd Signal")
plt.grid()
plt.tight_layout()
plt.show()

```



Question 6

Generate the delayed unit step signal:

$$u(t-3)$$

```

# Time axis
t4 = np.arange(-5,10,0.01)

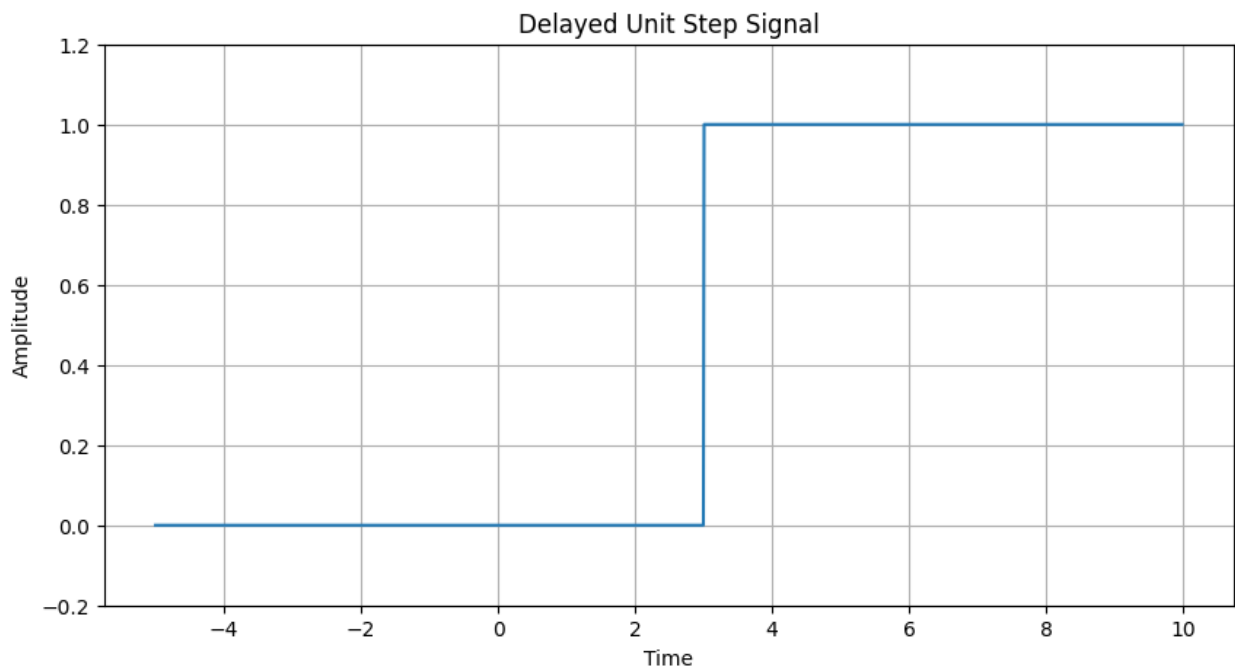
```

```

# Delayed step signal
u = np.where(t4 >= 3,1,0)

# Plotting
plt.figure(figsize=(10,5))
plt.plot(t4,u)
plt.title("Delayed Unit Step Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.ylim(-0.2,1.2)
plt.grid()
plt.show()

```



Advanced Problems

Question 1

Generate and compare:

- original signal
- shifted signal
- scaled signal
- reversed signal

```
# Original signal
x = np.sin(t)

# Shifted signal
x_shifted = np.sin(t-1)

# Scaled signal
x_scaled = np.sin(2*t)

# Reversed signal
x_reversed = np.sin(-t)

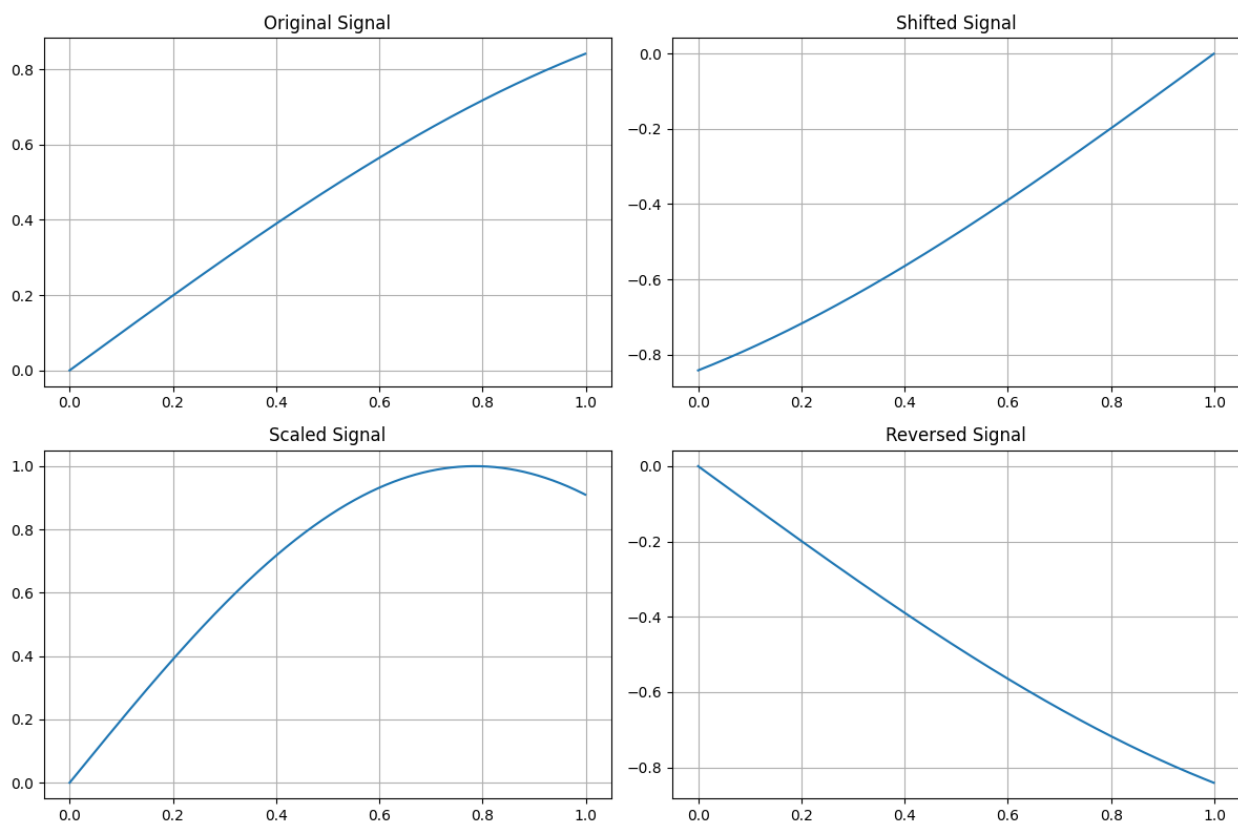
# Plotting
plt.figure(figsize=(12,8))

# Original
plt.subplot(2,2,1)
plt.plot(t,x)
plt.title("Original Signal")
plt.grid()

# Shifted
plt.subplot(2,2,2)
plt.plot(t,x_shifted)
plt.title("Shifted Signal")
plt.grid()

# Scaled
plt.subplot(2,2,3)
plt.plot(t,x_scaled)
plt.title("Scaled Signal")
```

```
plt.grid()
# Reversed
plt.subplot(2,2,4)
plt.plot(t,x_reversed)
plt.title("Reversed Signal")
plt.grid()
plt.tight_layout()
plt.show()
```



Comparison Table

Problem Type	Concept
CT Signal	Continuous representation
DT Signal	Sampled representation
Energy	Signal classification

Problem Type	Concept
Even/Odd	Symmetry
Step Signal	Switching operation
Transformations	Signal manipulation

Conclusion

The notebook demonstrated computational problem solving in signals and systems using Python.

